# Supplementary material: complete proofs

Source-Native Process Algebra and Initial-Coface Revision: Four-Tier Identity, Exact Evaluation Kernels, and Reversible Incremental Continuation

Jian Gao (高健)

---

This supplement gives complete self-contained written proofs of the results of §6.5, the end of §7.2, §8.5, §9, and §10.1–§10.3, together with the resolvent-comb, finite-window, and CPS1 instantiations of §10.4. The main text states the results and gives proof ideas; here each result is given its objects and domain, actual inputs and quantifiers, key lemmas, decisive derivation, and direct downstream uses, and is finally matched to its formal proof declarations. Notation follows the main text, and numbers of definitions, theorems, and equations refer to the main text. The proofs of §2–§5, §6.1–§6.4, early §7.1–§7.2, and §8.1–§8.4 are complete in the main text and Appendices A–C and are not repeated.

The "Formal correspondence" at the end of each section lists the Lean declarations carrying the result; pinned source commits and full paths appear in Appendices D.2–D.4 of the main text, and public reproduction entries appear in its "Code and Data Availability".

Pinned sources of this supplement: H = `e60a86058abcb74b7f2225f2383c182be8343785` (with C17/C18/C19–C21 increments X = `eec031c489`, Z = `453de27572`, AA = `d087a6b289`), AB = `fde5c14df36b6064c87064acd20830d3df0500bb`, AC = `b67aa48c75b3cd7738766ed40093bb3f7c1d7a1a`, AD = `aef0ad2a26199d9fd6da9bc9e7ec1a65636ef63f`, AE = `26b91feb6b507fdc3f43be9c429edb63fb9395ef`, PR = `e05558097256e86b537c019c8ca63cd449d6b2d2`, MF = `c62f3d25c42d4bf2b77ac009b49fc71cfd752f46`, MV2 = `71e94e8261d67cf4849a05eb14ac95aa2f7dcbb9`, MV3 = `9c73a630ce05bea062ec889b188f5105c0afb796`, MV4 = `ba7e274beb1de38900a6fc2c8c8dfe6cfad9d9aa`, MV5 = `df421ff600085cbc290a7ae946193961bed7c83b`, and `7e4da746f300b3796b8b18afb26cb0765c29afcb` (Homework); H0-S = `a1adcc9f4a06b872c381b8ad49513d6bf1cd621a` (public source-environment configuration). Appendices D.1–D.4 expand path abbreviations and list declaration entry points.

## S0 Common tools

**S0.1 Remaining count and execution.** Proposition 3.7 gives: every execution step decreases the remaining count by exactly 1; constants have no execution step; the left-to-right execution $\mathrm{exec}_\rho(e) : e \to \mathrm{const}(\mathrm{ev}_\rho e)$ has length $r(e)$; every execution trace is a derivation, so its endpoints evaluate equally (Proposition 3.6(1), below called **soundness**).

**S0.2 The pair language preserves the remaining count.** The pairing $e \mapsto \tilde{e}$ of Appendix A.2 replaces variables, constants, addition, linear and bilinear nodes one by one by nodes of the

same kind on pair values, so $r(\widetilde{e}) = r(e)$ (structural induction on $e$; every node contributes the same count), and

$$\mathrm{ev}_{(\rho,\varepsilon)}(\widetilde{e}) = (\mathrm{ev}_\rho(e),\ \mathrm{eff}_{\rho,\varepsilon}(e)). \tag{S0.1}$$

**S0.3 Relation image of a trace and the updated reading.** Let $t : e \to e'$ be an execution trace in environment $\rho$. Every execution step is a derivation certificate whose relation image is the difference of its endpoints; the relation image of the whole trace telescopes to $e - e'$. By soundness, $e - e' \in \ker(\mathrm{ev}_\rho)$. Given an increment $\varepsilon$, the update equation of Proposition 3.3 gives

$$\mathrm{ev}_{\rho+\varepsilon}(e - e') = \mathrm{ev}_\rho(e - e') + \mathrm{eff}_{\rho,\varepsilon}(e - e') = \mathrm{eff}_{\rho,\varepsilon}(e) - \mathrm{eff}_{\rho,\varepsilon}(e'). \tag{S0.2}$$

In particular, when $e' = \mathrm{const}(c)$ we have $\mathrm{eff}(e') = 0$, and the updated value is $\mathrm{eff}_{\rho,\varepsilon}(e)$.

**S0.4 Coimages and induced maps.** For a $\mathbb{Z}$-linear map $f : X \to Y$, write $\mathrm{Res}(f) := X/\ker f$ for its coimage and $\mathrm{can}_f : X \to \mathrm{Res}(f)$ for the quotient map; $\mathrm{Res}(f) \cong \mathrm{range}(f)$. If $f' \circ s = t \circ f$ (call $(s,t)$ a morphism from $f$ to $f'$), then $s(\ker f) \subseteq \ker f'$, so $s$ induces $\overline{s} : \mathrm{Res}(f) \to \mathrm{Res}(f')$ with $\overline{s} \circ \mathrm{can}_f = \mathrm{can}_{f'} \circ s$. Moreover $\mathrm{can}_f(x) = 0$ iff $f(x) = 0$.

**S0.5 Lifting residual.** Let $(s,t)$ be a morphism from $f : C \to D$ to $f' : C' \to D'$. For $c \in C$ its **fibre** is $\mathrm{Fib}_f(c) := \{x : f(x) = f(c)\}$. The map $s$ sends $\mathrm{Fib}_f(c)$ into $\mathrm{Fib}_{f'}(sc)$ and $\ker f$ into $\ker f'$. For $y \in \mathrm{Fib}_{f'}(sc)$ put

$$\mathrm{lift}(c,y) := [\, y - s(c)\,] \in \ker f' \big/ s(\ker f).$$

**Lemma S0.5.** $\mathrm{lift}(c,y) = 0$ iff there is $x \in \mathrm{Fib}_f(c)$ with $s(x) = y$.

*Proof.* If $y - s(c) = s(k)$ with $k \in \ker f$, take $x = c + k$; then $f(x) = f(c)$ and $s(x) = y$. Conversely, if $x \in \mathrm{Fib}_f(c)$ and $s(x) = y$, then $k = x - c \in \ker f$ and $y - s(c) = s(k)$. ■

**S0.6 Action-word model.** Let $X$ be an additive group with a family $(A_\ell)_{\ell \in L}$ of additive actions and an additive read-out $\mathrm{rd} : X \to Y$. For a word $w = \ell_1 \cdots \ell_n$ (acting first by $\ell_1$) write $A_w := A_{\ell_n} \cdots A_{\ell_1}$. Let $K := \{x : \forall w,\ \mathrm{rd}(A_w x) = 0\}$.

**Lemma S0.6.** (i) $K$ is closed under every $A_\ell$, so every $A_\ell$ descends to $\mathrm{Mod} := X/K$; (ii) $[x] = [y]$ iff $\forall w,\ \mathrm{rd}(A_w x) = \mathrm{rd}(A_w y)$; (iii) the map sending $[x]$ to the family of read-outs $(w \mapsto \mathrm{rd}(A_w x))$ is injective, so Mod is linearly isomorphic to the image of this family (the completion carrier of all word read-outs), and the isomorphism commutes with every action.

*Proof.* (i) If $x \in K$ then $\mathrm{rd}(A_w A_\ell x) = \mathrm{rd}(A_{\ell w} x) = 0$ for every $w$. (ii) By the definition of $K$ and linearity. (iii) By (ii); commutation is checked word by word on representatives. ■

**S0.7 Effect-history equation.** Let also $(U_\ell)$ be additive actions on $W$ and $M : X \to W$ an additive observation. For a point $x$ and a word $w$ let the total residual be $R(w,x) := U_w M(x) - M(A_w x)$ and the single-letter residual $r(\ell,x) := U_\ell M(x) - M(A_\ell x)$; define recursively

$$\mathrm{Acc}([\,],x) = 0, \qquad \mathrm{Acc}(\ell w, x) = U_w\, r(\ell,x) + \mathrm{Acc}(w, A_\ell x).$$

**Lemma S0.7.** $\mathrm{Acc}(w,x) = R(w,x)$; the history recording $(\ell_k, x_{k-1}, r(\ell_k, x_{k-1}))$ has length $|w|$.

*Proof.* Induction on the length of $w$:

$$\begin{aligned} R(\ell w, x) &= U_w U_\ell M(x) - M(A_w A_\ell x) \\ &= U_w(U_\ell M(x) - M(A_\ell x)) + (U_w M(A_\ell x) - M(A_w A_\ell x)); \end{aligned}$$

the first term is $U_w\, r(\ell, x)$ and the second is $R(w, A_\ell x)$. ■

**S0.8 Joint action-word carrier.** Let $I, H, Q$ be additive groups with three families of actions $(\alpha_\ell), (\eta_\ell), (\mu_\ell)$ and two additive read-outs $c : I \to H$, $m : I \to Q$. On $I \times H \times Q$ put $\theta_\ell := \alpha_\ell \times \eta_\ell \times \mu_\ell$ and $\mathrm{seed}(i) := (i, c(i), m(i))$. The **complete orbit carrier** $\mathcal{C}$ is the submodule spanned by all $\theta_w(\mathrm{seed}\, i)$.

**Lemma S0.8.** (i) $\mathcal{C}$ is the smallest submodule containing all seeds and closed under every $\theta_\ell$; (ii) the three projections $\pi_I, \pi_H, \pi_Q$ commute with the actions of words; (iii) the incidence element $\mathrm{inc}(w,i) := \theta_w(\mathrm{seed}\, i) - \mathrm{seed}(\alpha_w i)$ has $I$-component zero, and its $H$- and $Q$-components are $\eta_w c(i) - c(\alpha_w i)$ and $\mu_w m(i) - m(\alpha_w i)$.

*Proof.* (i) The generated submodule contains the seeds by definition; any submodule containing the seeds and closed under the actions contains every $\theta_w(\mathrm{seed}\, i)$ and hence their span. $\mathcal{C}$ itself is closed because the image of a generator is again a generator. (ii) $\theta_\ell$ is a product. (iii) Componentwise from (ii). ■

## S1 Theorem 6.10: birth and payment of a responsibility without a prior bearer

**Objects and domain.** Fix a world network $N$, an authoritative root on it with complete whole-ledger registration, an origin current $c$, and a reader that gives a complete environment–expression pair at every occurrence of the root at $c$. Let $o$ be the canonical occurrence at $c$, $(\rho, e)$ the output of the reader at $o$, $B = r(e)$, and $\Lambda = \Lambda_{\rho,e}$ the execution debt law of Definition 6.9.

**Inputs and quantifiers.** The theorem holds for all such data. No old responsibility row, registered inquiry, program table, table of future requests or empty-coverage certificate is supplied; the origin support stays fixed during the whole calculation.

**Construction.**

1. *Extended world and vocabulary.* The extended world $N[\Lambda]$ is given by Definition 6.3: its supports are Sup × Option(debt states). The currents of the vocabulary are the debt states of $\Lambda$, i.e., pairs $(e', t)$ with $t : e \to e'$ a paid execution trace; the initial current is $(e, \mathrm{id})$. The support of the current $x$ is $(\mathrm{supp}(o), \mathrm{some}\ x)$.
2. *Event algebra.* At the current $x = (e', t)$ the **source-selected action** is the output of the executor: a settlement if $e'$ is a constant, and otherwise the first step $s : e' \to e''$ of the execution of Proposition 3.7. The successor current $x^+$ is $x$ at a settlement and $(e'', t \cdot s)$ at an execution step. Compilation sends a settlement to the identity transport of $\Lambda$ (fourth item of Definition 6.9) and an execution step to a native write.

3. *Whole-ledger evolution.* whole$(x)$ sends every row on the support $(\mathrm{supp}(o), x)$ to $(\mathrm{supp}(o), x^+)$: the mathematical row row$(x)$ (the debt row of $\Lambda$ at state $x$) goes to row$(x^+)$, and every old row is transported unchanged. Destination and origin are mutually inverse.
4. *Patch, remainder and certificate.* The finite patch contains exactly one generated row, the mathematical row; all other rows are covered by the same-source transported remainder, whose occurrence is the evidence "this row belongs to the old ledger"—no finiteness of the old ledger is needed. The patch folds back to the whole-ledger evolution: $\mathrm{fold}(\mathrm{patch}(x)) = \mathrm{whole}(x)$. The restructuring certificate is generated by injectivity of the destination map of the whole-ledger evolution.
5. *Compiler, root and process.* The compiler emits patch and whole ledger at every current; the ledger root consists of source and compiler; the authoritative root and the living process are obtained by the standard construction of root registration. The state of the process at count $k$ is written $x_k$, with $x_0 = (e, \mathrm{id})$ and $x_{k+1} = x_k^+$.

**Key lemmas.**

**Lemma S1.1 (one-step budget).** $r(x^+) = r(x) - 1$ (natural subtraction); if $r(x) = 0$ then $x^+ = x$.

*Proof.* If $r(x) > 0$ then $e'$ is not a constant, the source-selected action is an execution step, and by S0.1 the count drops by one; if $r(x) = 0$ then $e'$ is a constant, the action is a settlement and the successor is unchanged. ■

**Lemma S1.2 (budget and history).** For every $k$: $r(x_k) = B - k$ and $|t_k| + r(x_k) = B$, where $t_k$ is the paid trace of $x_k$.

*Proof.* Induction on $k$. For $k = 0$, $t_0$ is empty and $r(x_0) = B$. Induction step: if $r(x_k) > 0$, the execution step lengthens the trace by one and decreases the count by one; if $r(x_k) = 0$, the state is unchanged and $B - (k+1) = 0$. ■

**Decisive derivation.**

1. *Birth.* By construction steps 2–4, every current emits exactly one event; compilation is the identity transport at a settlement and a native write at an execution step. The patch contains exactly one generated row; all old rows are kept as the remainder, and mutual inversion of origin and destination gives the restructuring certificate. The original support is unchanged (the first component of the support is always $\mathrm{supp}(o)$), and the law surface, old projections and type and value observations at the origin are kept as root readings.
2. *Payment.* The same-debt current at visit $k$ is row$(x_k)$; the destination equation $\mathrm{whole}(x_k)(\mathrm{row}(x_k)) = \mathrm{row}(x_{k+1})$ gives by induction on $k$ its lineage to row$(x_0)$. Its budget is $r(x_k) = B - k$ (Lemma S1.2). For $k < B$, $r(x_k) > 0$, the source-selected action is an execution step $s$, the same-debt target is row$(x_{k+1})$, and the budget drops strictly (Lemma S1.1); composed with empty histories on both sides it is a payment macro of Definition 6.6. The target budget is at most the current budget, so there is no refill; the continuation relation strictly decreases the budget and is well founded.
3. *Settlement.* At $k = B$ we have $r(x_B) = 0$, so the expression of $x_B$ is a constant const$(v)$ and a local settlement exists; a current of budget zero has no payment macro. The paid trace $t_B : e \to \mathrm{const}(v)$ has length $B$ (Lemma S1.2), and soundness gives $v = \mathrm{ev}_\rho(e)$. By Lemma

S1.1, $x_{B+1} = x_B$; the visit count keeps advancing, and visits of different causal depth are still distinguished in finite histories.

4. *Receipt.* By S0.3 the relation image of $t_B$ is $e - \mathrm{const}(\mathrm{ev}_\rho e) \in \ker(\mathrm{ev}_\rho)$; for every increment $\varepsilon$, its value under $\rho + \varepsilon$ is $\mathrm{eff}_{\rho,\varepsilon}(e)$. ■

**Direct use.** Proposition 6.11 applies this theorem with the whole-tree program as reader; Theorem 7.7 and every run of §9 use the same owner-free installation as their mathematical row, reading its value, charged history, original material and relation boundary.

**Formal correspondence.** `JS/OwnerFree/Source.lean` (`vocabulary`, `eventAlgebra`, `whole`, `mathEntry`, `destination_math`); `OwnerFree/Compiler.lean` (`remainder`, `patch`, `patch_fold`, `compiler`, `ledgerRoot`); `OwnerFree/Completion.lean` (`next_budget`, `budget`, `history_accounting`, `completed_value`, `completed_history_length`, `completed_next_state`); `OwnerFree/Payment.lean` (`lineage`, `debtCurrent`, `activePayment`, `wellFounded`, `no_refill`, `endpoint_no_paid`); `OwnerFree/Consumer.lean` (`value_source`, `paid_history`, `relation_boundary`, `updated_inverse_fibre`). Commit AB.

### S1.3 Theorem 6.12: Type completion receipts and continuing births

**Data and construction.** Fix a source program $\mathcal{P}$, recursive frames $\Phi_i$, and actual actions $a_i$, where $B_i$ is the remaining step count of the current paid-state expression. The receipt takes an actual starting index $i$ and returns—in `Type` rather than as an existential proposition—the dependent sum

$$\sum_{d:\mathbb{N}} \{s : \mathsf{Settlement}(\Phi_{i+d}.\mathrm{state}) \mid d \le B_i \ \wedge \ a_{i+d} = \mathrm{inl}\, s\}.$$

By strong recursion on $B_i$, eliminate $a_i$. A settled branch returns $d = 0$ together with its actual settlement $s$. A paid branch has $a_i = \mathrm{inr}\, p$, whose target **is** the mathematical state of $\Phi_{i+1}$. Since $B_{i+1} = B_i - 1 < B_i$, read the receipt $(d', s', ...)$ at that exact starting index and return $(d'+1, s', ...)$. The index identity $(i+1) + d' = i + (d'+1)$ transports the settlement event, and $d' \le B_i - 1$ gives $d' + 1 \le B_i$. This constructs a `Type` receipt at every starting index while retaining each actual branch equality.

**Distance equals budget.** Induct on $h \le B_i$. For $h < B_i$, the induction hypothesis gives $B_{i+h} = B_i - h > 0$. Because settlement requires a constant expression and zero budget, the actual action must be a payment, and the next frame satisfies $B_{i+h+1} = B_i - h - 1$. Consequently

$$B_{i+h} = B_i - h \quad (0 \le h \le B_i), \qquad a_{i+h} = \mathrm{inr}\, p_h \quad (h < B_i).$$

At the receipt endpoint the frame is settled, so $B_{i+d} = 0$; since $d \le B_i$, substituting into the formula above yields $d = B_i$. In particular, a zero-budget start returns distance zero without manufacturing a payment or bypassing settlement.

**Complete state and environment.** Each payment $p_h$ appends one execution step to the paid trace while preserving the registered input; induction yields the complete paid prefix together with every intermediate state, and soundness yields the endpoint constant value $\mathrm{ev}_\rho e$. The environment reader's registration-compatibility law guarantees that mathematical advances under a fixed input read back the same environment; applying that law step by step along the payment equalities

preserves the environment across $0 \le h \le B_i$ (the proof that distance equals budget does not depend on this environment law).

**Compilation and birth.** Unfolding the compiler at the endpoint equality $a_{i+B_i} = \mathrm{inl}\, s$, settlement invokes obligation admission and emits the active presentation of the born frame $\mathrm{nextBorn}(\Phi_{i+B_i}, \mathcal{P})$ as its canonical direct successor; the sealed run's node identity identifies this presentation with frame $i + B_i + 1$, so the admission successor and the next frame coincide as full states.

Finally set $u_0 = i$, $v_j = u_j + B_{u_j}$, and $u_{j+1} = v_j + 1$. Each $v_j$ carries its `Type` completion receipt above and satisfies $v_{j+1} \ge v_j + 1$; induction gives $v_j \ge i + j$, so for any tick $t$ taking $j \ge t$ yields $t \le v_j$. Thus the birth sequence is cofinal, and every segment $u_j, \dots, v_j$ consists of node values of the same recursive run at those indices. ■

**Formal correspondence.** The total-value writeback environment consumes the actual settlement stored in the receipt (S5.2), and cofinal stops attach each paid prefix to the same run (S5.3); declarations reside in MV2/MV3's `ACT/Completion.lean`, `Completion/Prefix.lean`, and `Completion/Births.lean` (`receipt`, `receipt_distance_exact`, `receipt_next`, `receipt_compiles`, and prefix/cover lemmas).

## S2 Theorem 8.12 and Theorem 7.7

### S2.1 Theorem 8.12: the actual action generates the next question

**Objects.** An inquiry frame $\Phi$: an inquiry state, the complete native program reading the root compiler, the raw input $(\rho, e)$ registered at the current visit, a reader $E$ giving the actual environment at every original occurrence, and a causal depth $k$. The mathematical state of the frame is a paid state $(e', t)$ of $\Lambda_{\rho,e}$; $\rho'$ is the reading of $E$ at the frame's occurrence and $\varepsilon = \rho' - \rho$.

**(1) Value of the request.** The residual request $e \ominus e' = e + \mathrm{linear}_{-\mathrm{id}}(e')$ is registered on $\rho + \varepsilon = \rho'$. By the definition of evaluation $\mathrm{ev}_{\rho'}(e \ominus e') = \mathrm{ev}_{\rho'}(e) - \mathrm{ev}_{\rho'}(e')$. Soundness gives $\mathrm{ev}_{\rho}(e) = \mathrm{ev}_{\rho}(e')$, so the old value is zero. The relation image of the paid trace is $e - e'$ (S0.3), whose value under $\rho'$ is by (S0.2) $\mathrm{eff}_{\rho,\varepsilon}(e) - \mathrm{eff}_{\rho,\varepsilon}(e')$, in agreement with the request's value. By S0.4 the canonical residual of this certificate under $\mathrm{ev}_{\rho'}$ vanishes iff its value vanishes, i.e., iff the request's value vanishes. At a settlement $e' = \mathrm{const}(\mathrm{ev}_{\rho} e)$ and the request's value is $\mathrm{eff}_{\rho,\varepsilon}(e)$.

**(2) Necessary birth.** $r(e \ominus e') = r(e) + (r(e') + 1) + 1$: the addition node contributes 1 and the linear node 1. This is positive, so the initial state is not a constant and the executor's first action is an execution step; in the birth compilation this step compiles to an obligation admission (the birth of a new responsibility), not to a payment of any old debt row.

**(3) Recursive frames and run.** Define the **rank** of a frame $\mathrm{rk}(\Phi) := (r(e),\ k + 1)$, ordered lexicographically. The next frame is determined by the current action:

- an execution step gives the same frame with depth increased by one, of rank $(r(e), k + 2)$;
- a settlement gives the born frame—old state the current inquiry state, program the successor program, registered input the residual request, depth zero—of rank $(r(e) + r(e') + 2,\ 1)$.

Both cases strictly increase the rank. The sequence of frames $\Phi_n$ ($\Phi_{n+1}$ the next frame of $\Phi_n$) is therefore strictly increasing lexicographically. The erasure of a frame determines its rank: the raw input is read back by the root's observation (erasure keeps vocabulary and visit, and the observation reads back the registered raw input), and the causal depth is read from the history of the erased visit. If $\Phi_m$ and $\Phi_n$ had the same erasure with $m < n$, they would have the same rank, contradicting strict increase. So erasure is injective. Validity of successors is checked branch by branch: an execution step uses the successor law of a direct answer, a settlement uses the successor law of an obligation admission, and both preserve the generated living law on the same root. The node of the existing canonical inquiry runtime at count $n$ is the presentation of $\Phi_n$: by induction on $n$, the activation successor of a node is the next frame. Every frame has a singleton query fibre (its only query is the frame's registered input), so no external table of future requests is needed.

**(4) Separation of payment and birth.** Restrict actual nodes to the debt process of the fixed root: the execution-step branch emits a same-debt payment receipt on the same mathematical row, whose whole-ledger destination equals the ledger evolution of the joint execution step (the reading of Theorem 6.10(2)); there is no payment macro where the budget is zero (Theorem 6.10(3)); the birth of the settlement branch is a new obligation admission. ■

**Formal correspondence.** `JS/Native/ResidualRequest.lean` (`expression_eval`, `old_value`, `budget`, `action_is_paid`, `updated_value`, `residual_zero_iff`); in `JS/Native/Restructuring/Inquiry/Continuation/`: `Policy.lean` (`rank`, `Before`, `next_progress`, `frames_forward`), `Inventory.lean` (`rank_eq_of_erasure`, `frames_erase_injective`), `Occurrence.lean` (`successor_valid`), `Runtime.lean` (`actual_node`, `macro_next`, `macro_next_preserves`), `Payment.lean` (`actual_paid_receipt`, `no_paid_of_zero`). Commit AB.

### S2.2 Theorem 7.7: shared run of the registered input and successor feed

**Objects.** The fixed data of Proposition 7.6: an old inquiry state (root, visit, audit protocol, calculus, query type, bearers and authority), a recognition installed on a complete complex inner-product space $\mathcal{H}$, the step at this visit and its disposition settle; in addition, the original inquiry input $\iota$ registered by the root at this visit: a projection position, its activation proof at this occurrence, a classifier equation and a query-type equation.

**(1) Registered activation.** Split by the output of settle.

*All-generated or joint-residual branch.* Proposition 7.6 gives a program package and the calculation inquiry state, whose root is the root with the calculation inquiry installed. On this root add a **query projection law**: the projection type is a singleton; at an occurrence $o'$ it is active iff $o'$ is the exact occurrence of this visit; the payload type is the calculation query; the projected value is the calculation query whose incidence is the original query of $\iota$. The new root $r_\iota$ is the original root with this projection added; the visit is unchanged. The registered input $\iota'$ takes this projection, the activation proof at the exact occurrence and the classifier equation (at the exact occurrence the classification takes the active branch by definition); the query-type equation holds by definition. The inquiry state $s_\iota$ has root $r_\iota$, keeps visit, audit protocol and calculus of the old state; bearers are read through the query's incidence; authority is the old authority lifted along the new projection face; the compilation program answers every query with the result face of the package embedded through the projection.

So $s_\iota$ compiles the query of $\iota'$ to an answer (by definition). The expression read by the answer face is the whole-tree program of the package, so the parser reads $T$ back (Theorem 3.9(3) and the tree equation of Proposition 7.6); the value is $\delta_{\tau(T)}$ (Theorem 3.9(1)); the source state is the paid state of the package. The compilation face of the old state for every original query $q$ is read back through the same embedding and still reads the canonical token of the old compilation.

*The other four branches.* The terminal branch returns the whole-ledger compilation of the root compiler at the source occurrence, and the three residual branches return the residual object itself, as in Proposition 7.6.

**(2) Shared run.** With $s_\iota$ as inquiry state and the constant function "return the raw input of the query of $\iota'$" as reader, construct the mathematical inquiry runtime (the owner-free installation of Theorem 6.10 plus the runtime of §8.5). The raw input of the owner-free installation is the raw input of this query; by Theorem 3.9(3) its expression reads back $T$. Theorem 6.10(3) gives a charged history of length $r(\mathrm{prog}_\kappa T)$, rewritten to $\beta(T)$ by Theorem 3.9(2), and the completed value $\mathrm{ev}(\mathrm{prog}_\kappa T) = \delta_{\tau(T)}$. The whole-ledger write-back and original material of the mathematical row at every count, and the actual query, answer and successor at every runtime offset, are the node equations of the mathematical inquiry runtime at that count and offset (Theorem 8.12(3)).

**(3) Successor feed.** The inputs are only the living root, the visit $v$, the recognition and the audit protocol. On nonterminal branches the step carries a successor succ and the equation $\mathrm{next}(v) = \mathrm{succ.current}$; put $v^+ := v.\mathrm{next}(\mathrm{succ})$. Recognition generates a step at $v^+$ and settle classifies it; the constructions of (1)(2) give again a feed and a run at $v^+$. The terminal branch returns only the whole-ledger compilation. Every nonterminal branch returns the preceding program package (the two aligned branches) or residual (the three residual branches) paired with the next run.

Whole-ledger write-backs and next currents of the two steps: the definition of a step contains $\mathrm{whole}(\mathrm{step}(v)) = \mathrm{generatedLedgerAt}(v.\mathrm{current})$ and $\mathrm{nextCurrent}(\mathrm{step}(v)) = \mathrm{generatedNextCurrentAt}(v)$, and the same holds at $v^+$. The successor's target occurrence equals the root's canonical occurrence at the current of $v^+$ (the target occurrence is the event the compiler emits for the next current), and history is a function of the occurrence, so the two histories are equal. ■

**Direct use.** The joint program of §9 (Theorem 9.2) takes the same recognition and the same successor as input; the default run of Theorem 9.8 consumes the next run of (3) at every visit.

**Formal correspondence.** `DJ/Input/{Source,Consumer}.lean`, `DJ/Input/Runtime/{Source,Consumer}.lean`, `DJ/Next/{Source,Consumer}.lean`; low source inquiry `JS/OwnerFree/Installation/Math/{Inquiry,Frame}/Source.lean`. Commit AC.

## S2.3 Theorem 8.14: same-occurrence differences, independent charge and the source environment square

**Two actual boundaries.** Take the successor packet generated by the source family together with its exact second-order successor occurrence; $b_\Psi$ is the pair lift of the terminal execution and $b_\mu$ is the complete boundary of the next compilation execution. Each execution trace witnesses membership in the relation closure of the next history without selecting occurrences by external

readouts. On the successor scope, let $r$ be the common root point, $\Psi$ the terminal value, $\mu = (\Psi, \sigma)$ the written value, and $c$ the constant embedding. The compiler's two affine boundary equations are

$$Q^+(b_\Psi) = r - c(\Psi, 0), \qquad Q^+(b_\mu) = r - c(\mu).$$

Hence $d = b_\Psi - b_\mu$ lies in the same relation closure, and linearity with abelian cancellation gives

$$Q^+(d) = (r - c(\Psi, 0)) - (r - c(\mu)) = c(\mu - (\Psi, 0)) = c(0, \sigma).$$

The actual environment on the successor scope reads this constant, so the independent difference query evaluates to $(0, \sigma)$. The history closure retains the responsibility provenance of both executions, and the updated value is read on the successor scope without assuming that the pre-update evaluation kernel remains the updated kernel.

**Independent charge of the coefficient program.** Write the finite support of $d$ in its support-list order as $(e_1, a_1), \dots, (e_k, a_k)$. Define $N_0(e) = \mathrm{const}(0)$ and $N_{n+1}(e) = e + N_n(e)$; expand a nonnegative integer coefficient $a$ to $N_a(e)$ and a negative coefficient $a = -(n+1)$ to $-N_{n+1}(e)$ (using one negation linear node). Induction on $n$ gives

$$\mathrm{ev}_\rho N_n(e) = n\,\mathrm{ev}_\rho e, \qquad r(N_n(e)) = n(r(e) + 1).$$

The integer expansion $E_a(e)$ therefore satisfies

$$\mathrm{ev}_\rho E_a(e) = a\,\mathrm{ev}_\rho e, \qquad r(E_a(e)) = |a|(r(e) + 1) + \mathbf{1}_{a<0}.$$

Setting $F([]) = \mathrm{const}(0)$ and $F((e, a) :: L) = E_a(e) + F(L)$, list induction gives

$$\mathrm{coeff}(d) = F(\mathrm{supportList}(d)), \qquad c(d) = \sum_{(e,a) \in \mathrm{supportList}(d)} (|a|(r(e) + 1) + \mathbf{1}_{a<0} + 1),$$

together with $\mathrm{ev}_\rho(\mathrm{coeff}(d)) = \mathrm{ev}_\rho d$ and $r(\mathrm{coeff}(d)) = c(d)$. An empty difference word has charge zero, while the subsequent residual birth retains its own positive budget. Theorem 6.10 executes this coefficient program, producing an execution trace of length $c(d)$, all $c(d) + 1$ debt states, and its own result $(0, \sigma)$ without borrowing any parent query's charge receipt.

**Projection installation and canonical direct successor.** Register this raw input, result, execution trace, and all offset material as a projection law on the root. Query incidence identifies that registration: answered compilation retains the query result, while residual-birth compilation triggers a new obligation admission at the same root and visit; the admission projection reads back the newly registered complete material and the whole-ledger remainder inherits every old projection, so effect, inverse residual, charge, execution trace, whole ledger, and canonical direct successor arise from a single compilation path, supplying the next raw input at packet $k+2$ without an external packet table.

**Feedback and the complete joint inventory.** Let $q$ be the full query of this source family, $q'$ its paid endpoint, $h = q \ominus q'$, $b_q := \delta_q - \delta_{q'}$ its formal trace boundary, and $\theta$ the action variable-substitution binding. Soundness gives $\mathrm{ev}_\rho h = 0$; substitution evaluation and the update identity

give

$$\mathrm{ev}_{\rho}(h[\theta]) = \mathrm{eff}_{\rho,\,\rho\circ\theta-\rho}(b_q), \qquad \mathrm{ev}_{\rho}(h + h[\theta]) = \mathrm{eff}_{\rho,\,\rho\circ\theta-\rho}(b_q),$$

where $\rho \circ \theta$ is the environment obtained by evaluating the substitution binding in $\rho$. Because $r(h) \geq 2$ and $r(h + h[\theta]) \geq 3$, this feedback expression debits strictly step by step and completes in its own remaining step count; S0.3 identifies the same value with the updated canonical coimage residual of $b_q$, and ordered mixed effects telescope step by step to the same complete result by S0.7. The joint inventory is defined as

$$\mathrm{seed}(H_{\mathrm{old}}, \mathrm{seed}(\mathrm{seed}(H_{\mathrm{dispatch}}, \mathrm{exposure}(T_q)), \mathrm{exposure}(T_{h+h[\theta]}))),$$

whose left and right parallel-seed inclusions retain the old inventory, every paid event of the query, and every newly paid feedback event for unified reading by the outer reader, inner query, and whole-ledger successor.

**Finite incoming configuration and environment-alignment square.** The finite incoming configuration sets the calculation reader to Incoming$(\Phi, D)$ (with query datum $D$): when the pair inventory is empty it returns $D.\mathrm{reader}(o)$ directly; when nonempty it selects an inventory expression or relation-word coefficient expression $g$ and returns

$$(D.\mathrm{reader}(o).\mathrm{environment},\ D.\mathrm{reader}(o).\mathrm{expression} + g).$$

In both branches the environment equals the query-reader environment by definition. Because the default action reader falls back to the query reader when unconfigured and agrees across both branches when configured with Incoming, the action-reader environment and query-reader environment coincide endogenously at every occurrence (moreover, Incoming never invokes a prior calculation reader recursively, so $g$ is added once while the query reader, decoder, and inventory preservation maps stay unchanged). Since the residual receiver has initial budget $B_{\mathrm{res}} = r(e) + r(e') + 2 \geq 2$ and retains budget $B_{\mathrm{res}} - 1 \geq 1$ after its first execution step, it stays strictly paid and closes the environment chain across write, birth, and the next query together with the alignment square. ■

**Formal correspondence.** The difference word and installation originate in PR and joint-carrier writeback in MF: `Cx/Profile/Finite/Reader/Difference/{Source,Inquiry/Source,Inquiry/Installation,Inquiry/Input,Inquiry/Receipt,Inquiry/Compiler,Inquiry/Actual,Actual}.lean`, `Profile/Producer{,/Consumer}.lean`, `Mixed/{Effect,Receipt,Consumer}.lean`, `Difference/Actual.lean`, and `Psi/TerminalTransport.lean`; the public source-environment configuration supplement resides in H0-S's `EffectInstalled.lean`, `GeneratedEnvironment.lean`, `FiniteCalculationInstallation.lean`, `FiniteIncoming.lean`, `ContextualFactory.lean`, `ActualNativeEnvironment.lean`, and `EnvironmentErasure.lean`.

# S3 Theorems 9.2 and 9.3

## S3.1 Theorem 9.2: one program carries the complete time, the common history and the action difference

**Objects.** A living root, a temporal visit $v$, a recognition and an audit protocol; $\mathrm{step}(v)$ lies in the all-generated or joint-residual branch, with successor $v^+$, history transition and pairing alignment; $T$ is the aligned constructor tree of Proposition 7.6.

**(1) Temporal codes.** A temporal visit of the root is given by one of three inductive histories: finitely reachable from the initial current, sealed cofinal, or reachable after cofinality. For each, encoding and decoding between the inductive types Hist, Post and the original reachability evidence are mutually inverse, defined and verified constructor by constructor. A temporal code is $c = (\text{current}, \text{history code})$; enc and dec take these encodings and decodings componentwise, so they are mutually inverse and enc is injective. The first component of $\mathrm{act}(c) = (\mathrm{rcpt}(c), \mathrm{nx}(c))$ contains $c$ itself, so act is injective. Successor: the successor of a temporal visit is given by the compiler's next current together with the extension of the history, and the encoding commutes with the extension constructor by constructor, so $\mathrm{enc}(v^+) = \mathrm{nx}(\mathrm{enc}\, v)$.

**(2) Joint program.** Nodes are pairs (aligned node of $T$, temporal code), and $T_J$ gives every node of $T$ the code $\mathrm{enc}(v)$. Results are five-tuples

$$\begin{gathered}(\text{effect fold},\ \text{receipt tree},\ \text{original material of both histories},\\ \text{evaluator inputs of both sides},\ \text{exposure plans of both sides}).\end{gathered}$$

The constructor $\kappa_J(n, \vec{y})$: its first component is the single-node fold of the original dependent effect $\mathrm{effectFold}(n_1, [\, y_{i,1} \,])$; its second component is the tree with root $\mathrm{act}(n_2)$ and the receipt trees of the children as subtrees; the last three components are the original material of both histories, the evaluator inputs of both sides and the exposures of both sides, independent of the children. The program is $P = \mathrm{prog}_{\kappa_J} T_J$ (Definition 3.8). The environment $\delta$ sends every node variable to its state point; $\delta'$ sends every node $n$ to $\delta_{n^+}$ with $n^+ := (n_1, \mathrm{nx}(n_2))$ (keeping $n_2$ when there is no successor).

*Value.* Mutual recursion on $T_J$ and child lists gives

$$\mathrm{ev}_{\delta'}(P) = \delta_{\mathrm{fold}_{\kappa_J}(T_J^+)},$$

where $T_J^+$ is the tree with every node replaced by $n^+$—the same as the proof of Theorem 3.9(1), with base points $n^+$. By S0.2 the value of $\widetilde{P}$ in the pair environment $(\delta, \delta' - \delta)$ is $(\mathrm{ev}_\delta P,\ \mathrm{eff}_{\delta,\delta'-\delta} P)$, and the update equation (Proposition 3.3) gives $\mathrm{eff}_{\delta,\delta'-\delta}(P) = \mathrm{ev}_{\delta'}(P) - \mathrm{ev}_\delta(P)$. Together the value is $(\delta_F, \delta_{F'} - \delta_F)$ with $F = \mathrm{fold}_{\kappa_J} T_J$ and $F' = \mathrm{fold}_{\kappa_J} T_J^+$.

*Components.* By induction on $T_J$: the first component of $F$ depends only on the first components of the nodes and of the children, and is exactly $\mathrm{fold}_{\mathrm{effectFold}}(T) = \tau(T)$; the second component of $F$ is the image tree of $T_J$ under act applied node by node, with root $\mathrm{act}(\mathrm{enc}\, v)$, which decodes back to $v$ by (1).

*Charge and payment.* By S0.2 and Theorem 3.9(2), $r(\widetilde{P}) = r(P) = \beta(T_J)$, and $T_J$ has the

shape of $T$, so $\beta(T_J) = \beta(T)$. Applying Theorem 6.10 to the reader $o \mapsto$ (pair environment, $\widetilde{P}$): the paid trace has length $\beta(T)$, every count $k < \beta(T)$ has a strict payment, the target budget is never refilled and the continuation is well founded.

*Inverse reading and successor.* At every count the expression kept by the installed face is $P$, and the parser reads back $T_J$ (Theorem 3.9(3)); forgetting the code component reads back $T$. The recovered root code decodes to $v$, and its generated next current is the root's generated next current at $v$. The update morphism $(\mathrm{id}, \mathrm{ev}_{\delta'})$ goes from $\mathrm{ev}_{\delta}$ to $\mathrm{ev}_{\delta'}$; on the source side take the relation words of the old trace, and as target fibre element the sum of the relation words of the old and new traces. By Lemma S0.5 the lifting residual vanishes iff the target fibre element is the image of an element of the old fibre; this residual is kept unchanged with the result.

**(3) Common history.** The source and target histories are histories of the same history law at two occurrences; events, supports, generator closures and relation closures of the common history are the unions of both sides. The two transitions embed the generators of each side by definition. The completion map sends a word $u$ of the generator closure of one side to the class of the same word in the common completion, so its image vanishes iff $u$ lies in the relation closure of the common history. The evaluators of the two sides are the faithful evaluations of the history law at the two occurrences, reading back the values of the original faithful faces on every generator; relation soundness is equivalent on both sides. The product of the completion carriers Src and Tgt carries the action and observation induced by the two transitions and forms an action model; pairing and dual action come from the scalar pairing of the histories; the measurement $\mathrm{Src} \times \mathrm{Tgt} \to \ell^2(\mathcal{H} \times \mathcal{H})$ together with a linear isometric evolution of that space forms joint integral–coherent action data.

**(4) Inverse criterion of the action difference.** Write $\partial_s, \partial_t$ for the differentials (measurements) of the two sides; the transition satisfies $\partial_t \circ \iota = \partial_s$.

*If* there is $r$ with $\partial_s r = \partial_s a_s$ and $\iota(r) = a_t$, then $d = \iota(r) - \iota(a_s) = \iota(r - a_s) \in \mathrm{range}\,\iota$, so the image residual vanishes, and $\partial_t d = \partial_s(r - a_s) = 0$, so the observation residual vanishes.

*Conversely*, a vanishing image residual gives $r_0$ with $\iota(r_0) = a_t$; then $d = \iota(r_0 - a_s)$ and the observation residual $\partial_t d = \partial_s(r_0 - a_s) = 0$, so $r_0 \in \mathrm{Fib}_{\partial_s}(a_s)$.

The computing program is a three-output expression: the difference $d$, its measurement, and its class in the cokernel; its remaining count is 25. Theorem 6.10 gives a paid trace of 25 steps, each a strict payment, with well-founded continuation, and whole ledger and next current kept at every count.

**(5) Literal successor.** The target occurrence of step($v$) is the event the compiler emits for the next current, equal to the canonical occurrence at the current of $v^+$, which is the source occurrence of step($v^+$). Histories, generator closures, relation closures, exposures and exposure trees are functions of (current, occurrence), hence pairwise equal; target words are carried to next source words along the equality of generator closures, with the underlying word unchanged. ■

**Formal correspondence.** `Occurrence/Temporal/{Source,Action/Source,Action/Inverse}.lean`; `Occurrence/Temporal/History/Common/{RootSource,Closure,Evaluator/Source,Action/Source}.lean`; `DJ/Joint/{Source,Laws,Consumer,TargetDifference,TargetProgramme,TargetConsumer}.lean`; `DJ/Next/{Material,Pursuit}.lean`. Commit AD.

### S3.2 Theorem 9.3: joint recovery by all actors

**Objects.** The data of Theorem 9.2 and a count. The source and target pairing unfolding trees have all pairing positions as nodes, each with an exposure $(M_p, A_p, U_p)$. Actors are members of the tree traces; the node Hilbert space is indexed by the positions of the traces (the same payload occurring repeatedly is counted at each position).

**(1) All-word model.** The joint carrier is the product of the carriers of the two sides; the action of a letter $\ell$ is the carrier action of that actor (source actors act on the first factor, target actors on the second, and "simultaneous" acts by the roots of both sides), and the actions of the root actors are the source and target exposure actions of the original step. The read-out rd is the product of the coarse reader and the measurements of all actors. Lemma S0.6 gives the quotient Mod, the fibre characterization and the completion equivalence; two elements of the completion carrier are equal iff their read-outs after all following words agree, which is S0.6(ii) stated on the completion side.

**(2) Ordered effect history.** Take $W = \mathcal{H}^{\text{actors}}$, let $U_\ell$ apply $U_p$ to the component of the selected actor and leave the others unchanged ("simultaneous" applies to the root components of both sides), and let $M$ be the measurements of the actors. Lemma S0.7 gives that the total residual is the sum of the step residuals transported by the later evolutions, with history length $|w|$; for a single letter the total residual is that actor's covariance effect $U_p M_p(x) - M_p(A_p x)$.

**(3) Node Hilbert.** The measurement meas : Mod $\to \ell^2(\text{positions}; \mathcal{H})$ takes at every position the feature of that actor (the actor's measurement read through the model). In $\ell^2$ the squared norm is the sum of the squared norms of the coordinates, so

$$\|\text{meas}(x)\|^2 = \sum_p \|M_p(x)\|^2.$$

The evolution of a letter on $\ell^2$ takes $U_p$ or the identity coordinatewise and is therefore a linear isometry. The node effect unfolds coordinatewise by the definition of the covariance coupling residual.

**(4) Future recovery and three faces.**

*Future recovery.* Let $x \in$ Mod be sent to zero by both the coarse reader and all future readers. The vanishing of all future $\mathcal{H}$ readers means: for every prefix bound and every word, every actor's measurement after the word agrees with that at the zero point. The vanishing coarse reader gives agreement of the coarse read-outs after every word. So $x$ and 0 have the same read-outs after every word, and (1) gives $x = 0$. The combined map is injective, Mod is linearly isomorphic to its image, and both composites are identities.

*Three faces.* With $I =$ Mod (acted on by the model advance), $H =$ the family of $\ell^2$ spaces over all prefix bounds (acted on by the evolution at each bound), $Q =$ the coarse model (acted on by the coarse advance), $c$ the future reader and $m$ the coarse restriction, form the complete orbit carrier $\mathcal{C} \subseteq I \times H \times Q$ of Lemma S0.8. The integral and coarse faces are the first and third components. The coherent face sends the second component (the family over all prefix bounds) to the inverse limit along restriction of prefix bounds: restriction at each bound commutes with evolution, so this is the completion map of a compatible family and commutes with the evolution

of every letter; each bound coordinate of the completion map is the value of the second component at that bound, so the coherent face vanishes iff the second component vanishes. An element on which all three faces vanish therefore has all three components zero, hence is zero: the three faces are jointly injective, give a double inverse on the image and commute with the action of words (S0.8(ii)).

**(5) Paid recovery.** For a point, a seed word, a history word and a query word $q$, the owner-free calculation executes the program generated by the query word, whose remaining count is $|q| + 3$ by the program-budget lemma; Theorem 6.10 gives a paid trace of length $|q| + 3$, and the computed value equals the next value (value-readback lemma). Applying the recovery map to the acted element reads back the next value as first component through the isomorphism of (4). The pair inventory enters the next birth as part of the born inventory, and the same debt is never refilled (Theorem 6.10(2)). ■

**Formal correspondence.** `AW/{Source,Complete/Source,EffectHistory/Equation,EffectHistory/Source}.lean`; `AW/NodeHilbert/{Source,Future/Recovery/Source}.lean`; `AW/NodeHilbert/Future/Wave/{Words/Kernel,Full/Source,Recovery/Source,Recovery/Consumer,Incidence/Consumer}.lean`. Commit AD.

### S3.3 Theorem 9.11: six faces of the same actual material

**Common material.** Take the paid material $M$ of the current frame, retaining occurrence $o$, environment $\rho$, expression $e$, paid endpoint $e'$, complete execution trace $T : e \to e'$, bearer, and environment increment $\varepsilon$. The relation words $\eta = T$.relationWords are generated step by step from the execution trace rather than inferred from terminal readouts; each step's relation image is $\delta_{e_h} - \delta_{e_{h+1}}$, telescoping to

$$\mathrm{rel}_\rho\, \eta = \sum_{h<|T|} (\delta_{e_h} - \delta_{e_{h+1}}) = \delta_e - \delta_{e'} =: b.$$

Let $H$ be the prefix history of this paid prefix; because every relation witness arises from that prefix, their sum lies in the relation closure $H$.relationClosure, defining the concrete relation element $\sigma = (b,\, b \in H.\text{relationClosure})$ whose projection into the history completion vanishes identically.

**Six faces and the complex.** The algebraic face retains $(e, e', e \ominus e')$ and the complete coefficient pairing; the combinatorial face retains the execution trace $T$ and every step's exposure position; the topological face is the prefix history together with its relation closure; and the logical face is the vanishing composite of the chain complex. Letting $d_1$ be relation-closure inclusion and $d_0$ history-completion projection gives

$$C_1 \xrightarrow{d_1} C_0 \xrightarrow{d_0} \widehat{C_0}, \qquad d_1(\sigma) = b, \qquad d_0 d_1(\sigma) = 0.$$

The relation face stores $\sigma$ and the cochain face stores this complex with the same boundary; all six faces are generated synchronously from the single material $M$, so their occurrence and raw-input readings coincide by definition.

**Effect and whole-word recovery.** By S0.3,

$$\mathrm{ev}_{\rho+\varepsilon} b = \mathrm{eff}_{\rho,\varepsilon} b = \mathrm{ev}_{\rho+\varepsilon}(e \ominus e').$$

In the quotient by the updated evaluation kernel, the canonical coimage class vanishes if and only if the right-hand side vanishes, and the standard coimage-to-range isomorphism reads back the native value. Because the whole-word recovery map reads every basis-word coefficient, its kernel vanishes strictly, yielding $\mathrm{rec}(\mathrm{can}(w)) = w$ for every formal word $w$ while the combinatorial face retains the full execution trace.

**Preservation across birth.** The admission's inventory-preservation map embeds this paid material into the born inventory while retaining the identical execution trace $T$; settled compilation supplies the full identity of the next root and current, along which transporting the installed raw input's projection, activation, classifier, and payload proofs yields the next installed query input and accompanying material. When the born frame has environment $\rho_B$, the prior registered request remains anchored at $\rho$ with updated increment $\rho_B - \rho$, retaining the old $(\sigma, b, T)$ with updated effect $\mathrm{eff}_{\rho,\rho_B-\rho} b$; reducing the new request at $\rho_B$ generates a distinct relation element $\sigma_B$, and the inventory stores both without conflating them. ■

**Formal correspondence.** MV3's `ACT/Receipt/Configured/Writeback/Successor/FourFace/{Source,Consumer,Native/Source,Native/Consumer}.lean` and `Operations/Inquiry/Cofinal/Native/{Source,Consumer,Registered/Source,Next/Source,Next/Consumer,Installed/Source,Installed/Consumer,Cofinal/Source,Cofinal/Consumer}.lean`.

## S4 Theorems 9.5–9.7

### S4.1 Theorem 9.5: universal source-program admission

**Objects.** An inquiry frame $\Phi$ and a source program $\mathcal{S} = (L, \mathrm{reader}, \ldots)$ (Definition 9.4). The shared runtime installs $\Phi$ and the reader of $\mathcal{S}$ as an inquiry state and answers its unique query; $s$ is this answered state, $o$ its current occurrence, and $\rho'$ the environment of the raw input of the query of the shared runtime's literal next state.

**(1) Material and value.** The material is assembled by definition: environment $\rho$ and expression $e$ are the output of the reader at $o$, the increment is $\varepsilon = \rho' - \rho$, the paid state $(e', t)$ is the result of the shared runtime at $o$, and the bearer is the bearer of this query at $s$. The registered input is the residual request of Definition 8.11, with environment $\rho + (\rho' - \rho) = \rho'$. Value and residual are given by S2.1(1).

**(2) Necessary admission.** If the initial state of the residual request were a settlement, its budget would be zero (the execution debt law has budget zero at settlements); but the residual request has budget $r(e) + r(e') + 2 > 0$ (S2.1(2)), a contradiction. So the source's first action is an execution step. The birth program is generated by the old state, the query, the registered input, the source packet, the authority and this step; on the root and visit of $s$, the compilation program sends the unique query to what the birth program generates on the exact temporal causal event—an obligation admission—and the equation holds by definition.

**(3) Target.** The admission target carries: a literal next current equal to (vocabulary, root, visit) of the target state; the first whole-ledger write; and, for every old projection, an outcome on the target equal (heterogeneously) to the original outcome. Validity of the successor is given by the general criterion "an active node preserves the generated living law under an admission successor", whose hypotheses are exactly the literal next current and the compilation equation above.

**(4) Born frame.** The born frame takes $s$ as old state, the residual request as registered input, the reader environment, depth zero, and as inventory the complete exposure of the paid trace. Its current state is by definition the admission target state. The runtime of Theorem 8.12 starts from it: the erasure of the initial node equals the erasure of the target presentation; the macro successor and preservation of the living law at every tick are the node equations of S2.1(3); no refill and well-foundedness are S2.1(4) and Theorem 6.10(2).

**(5) Next content.** By (S0.1) and the update equation, the sum of (old, effect) of the paid result equals the program's value in the updated environment:

$$\mathrm{ev}_{\rho}(e) + \mathrm{eff}_{\rho,\varepsilon}(e) = \mathrm{ev}_{\rho+\varepsilon}(e).$$

The environment of the born frame at every occurrence is given by the environment read-out of this value. The macro-history program satisfies $\mathrm{next}(\mathrm{hist}_k) = \mathrm{hist}_{k+1}$; the active environment of the initial frame is already the next value of the history of tick $k$, i.e., tick $k+1$, and the born frame advances one more tick, giving tick $k+2$. The born inventory is the side-by-side seed of the old inventory and the paid exposure, both of which are parts of it (left and right embeddings).

**(6) Native shift.** The native cursor shift of the query frame of the same stage gives a new frame; applying (1)–(4) to it gives compilation, first whole-ledger write, validity of the successor, literal successor, the run at every offset, no refill and well-foundedness; the shifted environment is the actual environment of the cursor shift, and effect and inverse reading are read from the residual request. ■

**Formal correspondence.** `ACT/Programme.lean`, `ACT/Receipt/{Source,Execution,Compiler,Consumer,Runtime}.lean`, `ACT/Target.lean`, `ACT/Receipt/Faces/{Source,Action,Consumer,Native/Source,Native/Consumer}.lean`; next content `AW/NodeHilbert/Future/Wave/Incidence/Environment/{Source,Consumer}.lean`. Commit AD.

### S4.2 Theorem 9.6: typed inventory crosses the admission

**Objects.** The setting of Theorem 9.5; the physical raw input $(\rho_P, e_P)$ (variables of physical type $\mathrm{Var}_P$) with its paid result, and the low-level raw input $(\rho_L, e_L)$ (variables in $L$) with its result. The binding is $\sigma : \mathrm{Var}_P \to \mathrm{Expr}_L$, $\sigma(x) = \mathrm{const}(\rho_P(x))$.

**(1) Binding preserves values.** Structural induction proves $\mathrm{ev}_{\rho_L}(e[\sigma]) = \mathrm{ev}_{\rho_P}(e)$: at a variable $\mathrm{ev}_{\rho_L}(\mathrm{const}\,\rho_P(x)) = \rho_P(x)$; at a constant both sides agree; addition, linear and bilinear nodes agree term by term by the induction hypothesis. Formal words extend linearly. Hence the value of the low-level query $(\rho_L, e_P[\sigma])$ equals that of the physical query. The substituted trace is the physical paid trace substituted along $\sigma$, starting at $e_P[\sigma]$, with its relation boundary substituted accordingly; the physical paid state is a constant and remains a constant after substitution. Every execution

trace ending at a constant has length equal to the remaining count of its start (Proposition 3.7(1)), so the substituted trace has length $r(e_P[\sigma])$.

**(2) Registration and readback of the inventory.** The complete material (both raw inputs and results, the binding, both written inventories and the substituted trace) is installed as one projection on the base root of the frame, with payload type the type of the material. The projection law of the universal admission target inherits the old projection law (Theorem 9.5(2)), and its outcome at this projection is heterogeneously equal to the original outcome; the equality of types is given by the same heterogeneous equality, and transporting the outcome along it to the material type gives the material itself.

**(3) Born inventory.** Every event of the physical inventory is carried into the low-level language by substitution along $\sigma$ and placed beside the low-level inventory to form the born inventory; the left and right embeddings ensure that every event of both sides is in it. The born frame runs as in Theorem 9.5; the next inventory extends the born inventory, and the coordinate map is injective under both actions mathNext and nextBorn and reads back every original event.

**(4) Inverse readback.** Index a word family $W$ by the events of the born inventory, with joint map $W \mapsto (\text{index} \mapsto (\text{ev}_\rho, \text{eff}_{\rho,\varepsilon})(W_{\text{index}}))$; the canonical residual of the actual word family vanishes iff its image vanishes (S0.4), and that image is the result value. The next inverse readback is the map induced by the update morphism (S0.4). At every tick of the cofinal stage, the pair value of a relation word in that tick's environment reads the value of the next tick's field at offset zero; the paid trace has length equal to the remaining count of the raw input. ■

**Formal correspondence.** `ACT/Receipt/Inventory/{Source,Laws,Installation}.lean`; `Inventory/Born/{Source,Consumer}.lean`; `Inventory/Born/Vector/Fibre/{Source,Consumer}.lean`; `Inventory/Born/Coordinates/Actual/{Source,Query}.lean`; `Coordinates/Actual/Cofinal/{Source,Consumer}.lean`. Commit AE.

### S4.3 Theorem 9.7: word–state pairing and paid prefixes

**Objects.** A runtime and its source; the old environment $\rho_s$ and increment $\varepsilon_s = \rho_{\sigma(s)} - \rho_s$ of a state $s$. The binding $\sigma$ of orbit variables satisfies two readbacks: $\text{ev}_{\rho_s}(\sigma x) = \rho_{\sigma(s)}(x)$ and $\text{eff}_{\rho_s,\varepsilon_s}(\sigma x) = \varepsilon_{\sigma(s)}(x)$.

**(1) Pairing.** On base points $\delta_s$, $\langle w, \cdot \rangle$ takes the stage inventory $(\text{ev}_{\rho_s} w, \text{eff}_{\rho_s,\varepsilon_s} w)$, extended uniquely and linearly by the universal property of free modules (Proposition 2.5); linearity in $w$ comes from linearity of evaluation and effect.

**(2) Covariance and coimage action.** On base points,

$$\langle w[\sigma], \delta_s \rangle = (\text{ev}_{\rho_s}(w[\sigma]),\ \text{eff}_{\rho_s,\varepsilon_s}(w[\sigma])) = (\text{ev}_{\rho_{\sigma(s)}} w,\ \text{eff}_{\rho_{\sigma(s)},\varepsilon_{\sigma(s)}} w) = \langle w, A\delta_s \rangle,$$

where the middle step is the substitution lemma—evaluation after substitution equals evaluation in the environment "variables take the values of their substitutes", and likewise for effects—and the two readbacks identify that environment with $\rho_{\sigma(s)}$ and $\varepsilon_{\sigma(s)}$. Both sides are linear on $\mathbb{Z}[S]$, so $\langle \sigma^*(\cdot), \cdot \rangle = \langle \cdot, A(\cdot) \rangle$ on the whole carrier. This says $(\sigma^*, A^*)$ is a morphism from the pairing to itself, which induces the action on the coimage by S0.4; the read-out formula on can$[e]$ holds by definition.

**(3) Iteration is time.** Induction on $n$: $n = 0$ is trivial; the step uses the base-point equation of (2) at tick $d$ and the induction hypothesis at $d + 1$. $\mathrm{act}^n[e] = [e[\sigma]^n]$ follows from commutation of the induced map with the canonical map.

**(4) Prefix charges.** The $i$-th prefix executes $\widetilde{e[\sigma]^i}$, and by S0.1 and S0.2 its trace has length $r(e[\sigma]^i)$; summing gives $\Sigma_n$. The batch query places the $n + 1$ terms in a vector-valued language: each term enters through one coordinate-embedding node and one accumulation node, so the remaining count is the sum of the terms plus $2(n + 1)$; execution length equals remaining count (Proposition 3.7(2)). Theorem 6.10 gives strict payment at every step and same-debt lineage.

**(5) Call recovery.** The actual query material of a call frame at count $c + i$ gives the environment there; the old values and increments of the snapshots needed are taken entry by entry from these environments (an increment is the difference of two adjacent ones). The batch environment so assembled equals the orbit batch environment coordinate by coordinate by definition, so the result values of the two queries are equal; the paid trace of every call has length equal to its remaining count (Theorem 6.10(3)), and summing gives call charge equal to call cost. ■

**Formal correspondence.** `Operations/Inquiry/Context/Native/Pairing/{Source,Orbit/Source,Orbit/Action,Orbit/Iteration,Orbit/Iteration/Batch,Orbit/Iteration/Current}.lean`; calls and history `⋯/Batch/Calls/{Source,Consumer,History/Source,History/Consumer}.lean` (full prefix in Appendices D.2–D.3 of the main text). Commit AE.

### S4.4 Theorem 9.9: complete recovery, dual inventories and order-preserving trace positions

**Whole-word recovery prior to projection.** Let $S$ be the state set of the sealed run, $\mathcal{F} = \mathbb{Z}[S]$ its free carrier, and $q$ the quotient map of the complete action-word model. Because the complete readout includes the whole-word reading at the empty action word, $\mathrm{rec}(q(w)) = w$ holds for every $w \in \mathcal{F}$. Define $O$ on basis states by reading the registered frame and extend linearly via $O(\delta_s) = \delta_{\mathrm{readFrame}(s)}$. At tick $i$, the runtime state stores the recursive index, so node equality and erasure injectivity yield $\mathrm{readFrame}(s_i) = \Phi_i$; combined with the model point action $Aq(\delta_{s_i}) = q(\delta_{s_{i+1}})$, this gives

$$O\mathrm{rec}(q(\delta_{s_i})) = \delta_{\Phi_i}, \qquad O\mathrm{rec}(Aq(\delta_{s_i})) = \delta_{\Phi_{i+1}}.$$

Projecting the scalar and pair inventories inside the frame then recovers the singleton word of the exact dual-inventory pair; if model points at ticks $i$ and $j$ agree, applying rec first recovers identical runtime states and strict monotonicity of frame rank forces $i = j$, so earlier paid states are never lost to premature frame truncation.

**Complete inventory handoff.** The raw input, value, execution trace, effect, inverse residual, decoder, whole ledger, and canonical direct successor of the current query are all supplied uniformly by the installed reader at that occurrence (the registered input when unconfigured, and the configured calculation reader when present); the stock program passes the entire material into universal admission while preserving both inventory update maps, and the `Type` completion receipt reads back the full born frame at distance plus one.

**Order-preserving trace-position injection.** Let $H_Q$ be the source-written exposure of the current query, $H_o$ the old written inventory, $H_m$ the migrated written inventory, and $H_P$ the physical written inventory. The combined written inventory and born inventory are defined by

$$H_C = \mathrm{seed}(\mathrm{seed}(H_o, H_Q), H_m), \qquad H_B = \mathrm{seed}(H_P, H_C).$$

Composing the inner right inclusion, middle left inclusion, and outer right inclusion across the three parallel-seed seams—each seam contributing offset 1 (total offset 3)—yields directly

$$H_B.\mathrm{trace}[\iota(j)] = H_Q.\mathrm{trace}[j], \qquad \iota(j) = j + |H_o| + |H_P| + 3.$$

Cancelling the shared natural-number offset $|H_o| + |H_P| + 3$ proves that $\iota$ is injective and strictly increasing ($i < j \Rightarrow \iota(i) < \iota(j)$), keeping equal-valued events at distinct trace slots strictly separated.

**Distinct old and new reading locations.** At settled admission, the inherited readout at the old initial visit retains the installed material whereas the successor readout at the new current delivers the next compiled query; each transports along its own full-current equality back to its respective reader without conflating the two distinct visits into one query. ■

**Formal correspondence.** MV2/MV3's `Operations/Inquiry/Context/Native/Frame/{Source,Stock,Stock/Return,Stock/Coordinates,Stock/Birth}.lean` (`actual_full_source_return`, `intoBorn_injective`, `intoBorn_read`, `intoBorn_strictMono`, and accompanying declarations).

## S5 Total-value writeback, cofinal continuation and whole-word source action

### S5.1 Theorem 9.8: the default run executes the source-generated request and its subtrees

**(1) Source-generated request.** Take the root and installed reader of the joint program of Theorem 9.2, form the source inquiry frame of the owner-free installation, and take as seed the zero unfolding of the generator of the registered expression. The source program of the default runtime has "the original request of this occurrence" as reader: at an occurrence $o$ the reader returns the request expression and environment generated for $o$. Apply Theorem 9.5 to this source program: the raw input of the query is the request read by the installed face (by definition); the result value equals the execution value of the request (the completed value of Theorem 6.10 together with soundness of ev); the paid trace has length equal to the remaining count of the registered input (Theorem 6.10(3)); by Theorem 9.5(5) the born pair inventory consists of the previously written part and the part paid now; the actual query, answer and successor at every count are node equations of the runtime. The ledger roots agree by construction. In the branch version the core and relation expressions are the two coordinates of a binary batch, and the trace and substituted trace of the batch execution enter the initial inventory.

**(2) Subtrees enter the main expression.** The main expression is a sum of three terms: the original query expression, the subtree at the old code and the subtree at the next code, each entering the output type through a coordinate embedding. The subtree at a code is the zero

constant if recognition has no successor at the visit of that code, and otherwise the embedding of the raw input of the subprogram at that code. Hence

$$\mathrm{ev}(\mathrm{main}) = (\mathrm{ev}(\mathrm{query}),\ \mathrm{ch}(c_{\mathrm{old}}) + \mathrm{ch}(c_{\mathrm{next}})),$$

where the first component of a subtree value is zero. Remaining counts add term by term, plus two additions and one linear embedding, three combining nodes in all: $r(\mathrm{main}) = r(\mathrm{query}) + c_{\mathrm{old}} + c_{\mathrm{next}} + 3$. Paid trace length: the trace of the owner-free installation has length equal to the remaining count, and the query's own paid trace has length $r(\mathrm{query})$; substituting gives the claim. The executions of the two subtrees have length equal to their own remaining counts (Proposition 3.7(2)). Events of the old inventory and of the new payment are kept by the embeddings of the side-by-side seed; the sum of the old and next actor components is the state point of this tick's source operation, read back through the complete actor carrier.

**(3) Subtree environments.** At a supplied occurrence the subtree environment is defined by the read-out of the original binding in that environment; the environment of the born frame is the actual updated environment (Theorem 9.5(5)), and the subtree environments are read from it by the same binding. Scalar and pair inventories are kept as in (2); the initial node and the input at every tick of the runtime are the runtime's defining equations. ■

**Formal correspondence.** `ACT/Inventory/Pair/Residual/Joint/Past/NextObservation/Request/Acted/{Source,Consumer}.lean`; `DJ/{Joint,Branch}/Primary/{Source,Consumer}.lean`; `DJ/Branch/JointQuery/Actor/{Source,Embedding,Action/Source,Operation/Source,Operation/Consumer}.lean` and `Actor/Operation/Query/Inventory/{Language,Source,Consumer,Action/Source,Action/Consumer,Live/Source,Live/Consumer}.lean`; default successor `DJ/Next/{Source,Consumer}.lean`. Commit AE.

## S5.2 Theorem 8.15: actual total value, per-receipt decoder regeneration and birth writeback

**Same-occurrence action material.** At occurrence $o$, the action reader supplies raw input $(\rho, e)$, while the settled result supplies paid endpoint $\mathrm{const}(p)$, the complete execution trace, and old paid value $p = \mathrm{ev}_{\rho} e$. The action material $M$ retains these data together with the successor environment increment $\varepsilon$; on the updated environment $\rho + \varepsilon$, the residual request $N(M) = e \ominus \mathrm{const}(p)$ satisfies

$$\mathrm{ev}_{\rho+\varepsilon} N(M) = \mathrm{ev}_{\rho+\varepsilon} e - p =: n, \qquad p + n = \mathrm{ev}_{\rho+\varepsilon} e.$$

The first identity unfolds subtraction and the second uses abelian cancellation alone: $p$ and $n$ are the old paid query value and the completed residual value of the current pass, requiring no externally supplied post-update target total.

**Total-value writeback on the settled branch.** Keeping the low-level program's source, query and calculation readers, effect/inverse reading, whole ledger, and dual-inventory preservation maps unchanged, set the next-environment reader to $\mathrm{write}(p + \nu(o'))$, where $\nu$ vanishes on intermediate debit branches and returns the settled completion value on the settlement branch. Along the frame runtime, the settlement event $o_B$ selected by the `Type` receipt of Theorem 6.12

satisfies $\nu(o_B) = n$, so unfolding the born-environment field of settled compilation yields

$$\mathrm{env}(\mathrm{nextBorn}) = \mathrm{write}(p + \nu(o_B)) = \mathrm{write}(p + n).$$

Intermediate debit branches advance only the mathematical paid state without invoking birth-environment writeback; in the finite incoming source family of S2.3, the environment-alignment square also aligns the query-reader environment strictly with this action-material environment.

**Cross-packet decoder regeneration.** Let $P_j$ be packet $j$, let the receipt supply born frame $B_j$, and let packet $P_{j+1}$ form its new seed from $B_j$'s complete material and registered expression. The configuration at packet $P_{j+1}$ re-reads its own action result $p_{j+1}$ and residual completion function $\nu_{j+1}$ to install the fresh decoder $\mathrm{write}_{j+1}(p_{j+1} + \nu_{j+1})$; repeating the argument above on $P_{j+1}$ gives

$$\rho_{B_{j+1}} = \mathrm{write}_{j+1}(p_{j+1} + n_{j+1})$$

without reusing the prior packet's old value $p_j$, so induction on $j$ regenerates an independent decoder at every receipt while preserving both inventories. ■

**Constant zero-effect positive control.** For the constant configured query $\mathrm{const}(1, 0)$, the old paid value is $p = (1, 0)$ and the current update effect is $n = 0$; the born environment writes back $(1, 0)$ and the next paired generation retains nonzero paid value $((1, 0), (0, 0))$, confirming that native effect $n$ cannot substitute for the complete total value $p + n$.

**Formal correspondence.** MV3's `ACT/Receipt/Configured/Writeback/{Source,Consumer}.lean` and `Writeback/Successor/{Source,Consumer}.lean` (`old_paid_value`, `native_value`, `updated_total`, `born_environment`, `successorPacket`, `second_decoder`, `second_writeback`), together with the resolvent-comb instantiations `Feedback/ClozelOriginalK{ConfiguredPairWriteback,SourceSuccessor}.lean`.

## S5.3 Theorem 9.10: cofinal admission stops and complete intermediate-segment cover of a single run

**Two-stage runtime representation.** The runtime stage is a dependent sum: an admission state retains packet grade, datum, configuration, and language, whereas a continuation state additionally carries the current paid inquiry frame. One-step advance sends an admission state into continuation; on an intermediate debit branch of continuation it keeps packet grade and material while advancing the mathematical frame, and on a settled branch it connects the born frame to the next packet seed and returns to admission—forming a single source-generated function with no external schedule.

`Type` **distance to the next admission.** At any tick $t$, split by stage: if already in admission, return distance 0; if in continuation, invoke the recursive-frame `Type` receipt of S1.3 to obtain the exact remaining debit count $B$ together with all $B$ ticks of the paid prefix, and add 1 tick for the settlement-to-admission transition to obtain total distance $B+1$. Ordering states lexicographically by (inventory size, causal depth $+1$), admission and settlement strictly grow the inventory while intermediate debits preserve the inventory and strictly increase depth, so source erasure is injective across all runtime ticks.

**Cofinal stops and segment cover.** Letting $d(t)$ denote the distance above, define recursively $t_0 = d(0)$, $\ell_j = 1 + d(t_j + 1)$, and $t_{j+1} = t_j + \ell_j$. Both endpoints lie in the admission stage, every intermediate inquiry frame at $t_j + h$ $(0 \le h \le \ell_j)$ is retained in full, and

$$t_j + \ell_j = t_{j+1}, \qquad t_{j+1} \ge t_j + 1, \qquad t_j \ge j.$$

For any tick $t$, taking $j = t$ gives $t \le t_j$ (strict cofinality), and choosing the first stop beyond $t$ covers $t$ within the preceding segment; packet grade $j$ increments by 1 solely at admission transitions and never impersonates the underlying runtime tick.

**Complete field-word advance.** Writing $W_t$ for the complete field word at tick $t$, the one-tick state-point action satisfies $A_{\mathcal{R}} W_t = W_{t+1}$; induction on $h$ gives $A_{\mathcal{R}}^h W_t = W_{t+h}$, and setting $h = \ell_j$ proves $A_{\mathcal{R}}^{\ell_j} W_{t_j} = W_{t_{j+1}}$, so whole words on the free carrier, cofinal-point injection, and word–state duality commute strictly across the same span. ■

**Formal correspondence.** MV3's `F/Admission.lean`, `F/SourceFamily/{Runtime,Activation/Source,Activation/Stage,Activation/Admissions}.lean` (prefix `F/` expanded in Appendix D.3), and `Operations/Inquiry/Cofinal/{Source,Consumer}.lean` (`source_span`, `strictMono`, `cofinal`, `cover`).

## S5.4 Theorem 9.12: dependently typed direct-sum whole-word source action and maximal future-invisible kernel model

**Per-packet language and receiver pairing square.** From S3.3 and S5.3 take the installed raw input and packet equalities at stop $j$, whose complete formal-word module $\mathcal{W}_j$ carries the paired readout

$$d_j(u) = (\mathrm{ev}_{\rho_j} u, \mathrm{eff}_{\rho_j, \varepsilon_j} u).$$

Pair-lifting expressions in $\mathcal{W}_j$ node by node and transporting along the next-packet language equality defines the stepwise source-word action $S_j : \mathcal{W}_j \to \mathcal{W}_{j+1}$; extending the paired evaluation identity of Appendix A.2 linearly yields the receiver square

$$\mathrm{ev}_{\rho_{j+1}^{\mathrm{receiver}}}(S_j u) = d_j(u)$$

without assuming equal environments $\rho_{j+1} = \rho_j$ across distinct packet grades.

**Finitely supported direct sum and inventory preservation.** Scalar relation-inventory members enter the born relation inventory via pair lifting, while the pair inventory and paid prefixes remain stored across both debit and birth steps. On the finitely supported direct sum $\mathcal{W} = \bigoplus_j \mathcal{W}_j$, define

$$S(w) = \sum_{j \in \mathrm{supp}(w)} \mathrm{in}_{j+1}(S_j w_j), \qquad D_j(w) = d_j(w_j).$$

Because shifting $j \mapsto j + 1$ preserves finite support, $S$ is well-defined and $\mathbb{Z}$-linear; across finite prefixes $0, \dots, b$, the site readouts $D_j$ commute with deleting the last site and generate the joint completion carrier.

**All future words and the maximal invisible invariant submodule.** Over the single-letter alphabet $\{*\}$ acting by $S$ ($S_{[]} = \mathrm{id},\ S_{a*} = S \circ S_a$), define

$$K_{\mathrm{fut}} = \bigcap_{a \in \{*\}^{<\omega}} \bigcap_{j} \ker(D_j \circ S_a).$$

For any $w \in K_{\mathrm{fut}}$ and any $a, j$, $D_j(S_a(Sw)) = D_j(S_{*a}w) = 0$, so $S(K_{\mathrm{fut}}) \subseteq K_{\mathrm{fut}}$ and the quotient map $q_{\mathrm{fut}} : \mathcal{W} \to \mathsf{M}_{\mathrm{fut}} := \mathcal{W}/K_{\mathrm{fut}}$ induces a unique model action $\bar{S}$ and descended readouts $\bar{D}_{j,a}$ satisfying

$$\bar{S} q_{\mathrm{fut}} = q_{\mathrm{fut}} S, \qquad \bar{D}_{j,a} q_{\mathrm{fut}} = D_j S_a.$$

Two elements share a quotient class if and only if their readouts agree after every allowed future word at every site. If a submodule $L \subseteq \bigcap_j \ker D_j$ satisfies $S(L) \subseteq L$, induction on word length gives $S_a(L) \subseteq L \subseteq \bigcap_j \ker D_j$, whence $L \subseteq K_{\mathrm{fut}}$, proving maximality; specializing to the empty word $a = [\,]$ gives $K_{\mathrm{fut}} \subseteq \bigcap_j \ker D_j$, projecting naturally back to the present joint completion and commuting with $\bar{S}$. ■

**Formal correspondence.** MV3's `Operations/Inquiry/Cofinal/Native/Installed/Cofinal/Tower/{Source,Consumer,Action/Source,Action/Consumer,Action/Common/Source,Action/Common/Consumer}.lean` and `Operations/Observed/Words/{Kernel,Model}.lean`.

## S5.5 §10.4: complete resolvent-comb $R$ calculation, total-value feedback and Mellin pricing identity

**Pre-emitter complete calculation and installation.** Right-half-plane nontrivial-zero observation supplies $\zeta$ (with $\Re\zeta > 1/2$, so resolvent parameter $z = \zeta/2$ satisfies $\Re z > 1/4$), exact $\Xi$-order $m$, and initial comb vector $q_0$. Setting $e_0 = x,\ e_{k+1} = \mathrm{linear}_R(e_k)$, induction on $k$ gives

$$\mathrm{ev}_\rho e_k = R^k(\rho(x)), \qquad r(e_k) = k + 1.$$

Executing at $\rho(x) = q_0$ produces computed value $v = R^m q_0$ and its complete execution trace of length $m + 1$; prior to emission, the sealed facade installs this result law at the same occurrence while the projection remainder preserves the underlying whole ledger and physical successor intact.

**Endogenous increment generation from the computed value.** Setting $\rho^+ = \mathrm{constEnv}(v)$ and environment increment $\delta = \rho^+ - \rho$, the residual request on the paid material evaluates to

$$\mathrm{ev}_{\rho+\delta}(e_m \ominus \mathrm{const}(v)) = R^m v - v,$$

which the `Type` receipt of S1.3 and additive decoder $(a, b) \mapsto a + b$ of S5.2 write back into the born environment as $\mathrm{constEnv}(v + (R^m v - v)) = \mathrm{constEnv}(R^m v)$.

**Full-$R_z$ execution before orthogonal split and Mellin pricing.** Let $R_z$ be the ambient resolvent operator and $P$ the orthogonal projection onto the closed subspace. On paired input $(v, 0)$, the three-step program executes variable bind $\to (v, 0)$, full linear resolvent $\to (u, 0) = (R_z v, 0)$, and orthogonal split $\to (p, r) = (Pu, (1 - P)u)$ (3 primitive debits in total, retaining intermediate node $(u, 0)$ and the decomposition $p + r = u$ inside the trace). On $\Re z > 1/4,\ 1/4 < \Re w < 1/2$, the

completed ambient Mellin functional $L_w$ commutes with the Bochner integral of $R_z$, and dilation reduces the kernel to an exponentially decaying integral with $\Re(1/2 - z - w) < 0$:

$$L_w(R_z v) = -\int_0^\infty e^{(1/2-z-w)h}\, dh\; L_w(v) = -\frac{L_w(v)}{z + w - 1/2}.$$

Cancelling normalization via $\Gamma_{\mathbb{R}}(2w) \neq 0$ yields physical heat readout $H_w(v) = 2L_w(v)$ and remainder readout $M_w(r) = L_w(r)$, so applying $2L_w$ to $p + r = R_z v$ gives the complete pricing identity

$$H_w(p) + 2M_w(r) = -\frac{H_w(v)}{z + w - 1/2}.$$

The next packet continues from physical born value $p$ to generate $(PR_z p, (1-P)R_z p)$ (even when the current residual vanishes, S5.2 writes back the full old paid value and regenerates the decoder per packet). ■

**Formal correspondence.** The calculation face originates in MV2 and the total-value feedback and full-$R$ consumers in MV3: under `NoIslandNoMagic/CanonicalRiemann/GeneralizedDual/CoPoissonEnergy/Muntz/Cokernel/CenteredGram/JointAction/Runtime/`, `Material/Calculation/ClozelOriginalKCombSourceCalculation.lean`, `Root/ClozelPairedOmegaEffectRowRoot.lean`, `Facade/ClozelPairedOmegaEffectRuntimeFacade.lean`, and `Feedback/ClozelOriginalK{ReceiptSource,ReceiptFeedback,PhysicalPricing,PhysicalAction,PhysicalFeedback,RegisteredSourceFeed,ConfiguredPairWriteback,SourceSuccessor}.lean`.

### S5.6 §10.4: 37-primitive whole-atom smooth system and energy-reserve dispatch

**Primitive input and `Type 0` body generation.** For a fixed 37-entry primitive input $I$, the finite in-source program executes translation, repair, resource response, and chemical exchange in sequence; the dependent type $\mathsf{InputBodyAt}(I)$ performs exhaustive constructive elimination over all outcome branches, and the positive completion proof on the fixed source instance selects the successful branch, generating from value fields a `Type 0` body $B : \mathsf{InputBodyAt}(I)$ carrying DNA, full coding, proteins, nuclear and electronic fields, resource account, carbamoyl-phosphate reaction-site focus, open responsibility, full inventory, and written history. On native current $(\text{phase}, I, B)$, each tick writes and verifies the canonical direct-successor identity

$$\text{next}(\text{phase}, I, B) = (\text{phase} + 1, I, \text{bodyNext}_I(B)),$$

so induction on ticks identifies the running body with iterated $\text{bodyNext}_I$ and aligns delivery readouts field by field with the root current, emitted write, and target successor.

**Full smoothness and dyadic time refinement.** At the post-step state, the electronic occupied-state matrix and nuclear configuration undergo half-step Cayley updates, while the full nuclear–electronic energy includes all nuclear repulsion terms, the electronic trace, and two-body quartic Coulomb terms; the four local/stored cases of Gaussian nuclear integrals admit differentiation under the integral sign, and separated nuclear configurations keep pairwise internuclear distances strictly positive, establishing full smoothness directly from the present fields without externally supplied future force fields. Writing $\pi(t)$ for the candidate step's energy-

difference price, as $t \to 0$ the half-step occupied state tends to the initial state, forces remain continuous, and nuclear displacements tend to zero (preserving separation), so energy continuity yields

$$\pi(0) = 0, \qquad \lim_{t\to 0} \pi(t) = 0.$$

Dyadic refinement halves the timestep $t_k = t_0\, 2^{-k}$ while keeping fields, inventory, and accounts fixed, reaching readiness at sufficiently large $k$.

**Energy-reserve dispatch of finite `Type` dispositions and tick-by-tick fulfilment.** When reserve $A > 0$, $\lim_{t\to 0} \pi(t) = 0$ guarantees a finite ready index $k$ with $\pi(t_k) < A$; `Nat.find` endogenously selects the first ready index $k_*$ satisfying $\pi(t_{k_*}) \le A$ and returns a paid `Type` disposition retaining $k_*$, the post-step state, and the updated ledger (every $k < k_*$ is an unpaid refinement step preserving total charge/electron count and total energy $E+A$). When $A = 0$, taking the first ready index $k_*$ and testing $\pi(t_{k_*}) \le 0$ versus $> 0$ dispatches either a paid disposition or an explicit energy-shortage disposition retaining the positive-price witness, never fabricating payment out of zero reserve. By induction on step count, the refinement-step body sequence agrees heterogeneously with the canonical direct successors of the sealed runtime, running the same authoritative root continuously across every evolutionary tick. ■

**Formal correspondence.** MV3's `NoIslandNoMagic/Consciousness/Immortality/Embodied/Molecule/CPS1Personalized2025/` (`Source/NativeRegistration/{Source,Programme,Registration,Positive,BodyPositive}.lean`, `Root/{NativeCurrent,Base,Installation}.lean`, `Delivery/Runtime.lean`, `Runtime/{RegisteredBody,BodyPaid,BodyFunction}.lean`) and `Lean/DomainProofs/CPS1ResourceExecution/CPS1MaterialIncidence/` (`NativeRoot{FromInput,Data,MaterialData,NextLaw}.lean`, `Native{RawRegularity,FullSmooth,BodyPaidSource,BodyFunction}.lean`).

## S6 Complete proofs for reversible incremental continuation and cross-horizon methods (Theorems 10.1–10.6, C46–C54)

### S6.1 Theorem 10.1 (C46): universal semantic-zero congruence and writable requests in canonical next

**Objects and quantifiers.** Fix an arbitrary sort set $\mathcal{K}$, additive abelian group family $W$, variable family $X$, graded source-program family `Factory`, initial inquiry frame $\Phi_0$, program configuration cfg, language equality $L_{\mathrm{cfg}} = X$, and any actual cofinal packet ordinal $j \in \mathbb{N}$. For any value-group carrier $V$, define the **all-environment semantic-zero submodule**

$$\mathcal{Z}_{\mathrm{sem}}(V) := \{ w \in \mathrm{Fm}_{\mathbb{Z}}(\kappa) \mid \forall \rho \in \mathrm{Env}(V, X),\ \mathrm{ev}_\rho(w) = 0 \}. \tag{S6.1}$$

By $\mathbb{Z}$-linearity of evaluation on formal words, $0 \in \mathcal{Z}_{\mathrm{sem}}$ and $\mathcal{Z}_{\mathrm{sem}}$ is closed under addition, negation, and integer scaling, forming a submodule.

**Key lemma and decisive derivation.**

**Lemma S6.1 (Universal inventory vanishing and pair-lift closure).** If $w \in \mathcal{Z}_{\mathrm{sem}}(V)$, then: (i) for every old environment $\rho$ and increment $\varepsilon$, the update inventory satisfies $U_{\rho,\varepsilon}(w) = (\mathrm{ev}_\rho(w), \mathrm{eff}_{\rho,\varepsilon}(w)) = (0, 0)$; (ii) the pair-lifted word satisfies

$\widetilde{w} \in \mathcal{Z}_{\mathrm{sem}}(V \times V)$; and (iii) linear transport along any packet equality or variable-language equality preserves membership in $\mathcal{Z}_{\mathrm{sem}}$.

*Proof.* (i) By the update equation $\mathrm{ev}_{\rho+\varepsilon}(w) = \mathrm{ev}_{\rho}(w) + \mathrm{eff}_{\rho,\varepsilon}(w)$ (Proposition 3.3), $w \in \mathcal{Z}_{\mathrm{sem}}$ gives $\mathrm{ev}_{\rho+\varepsilon}(w) = 0$ and $\mathrm{ev}_{\rho}(w) = 0$, cancelling to $\mathrm{eff}_{\rho,\varepsilon}(w) = 0$. (ii) Any paired environment $\tilde{\rho} \in \mathrm{Env}(V \times V, X)$ decomposes pointwise as $(\rho, \varepsilon)$ with $\rho(x) := \pi_1(\tilde{\rho}(x))$, $\varepsilon(x) := \pi_2(\tilde{\rho}(x))$; extending (S0.1) linearly gives $\mathrm{ev}_{(\rho,\varepsilon)}(\widetilde{w}) = U_{\rho,\varepsilon}(w) = (0, 0)$. (iii) Case elimination on the equality reduces transport to the identity linear isomorphism. ■

**(1) Exact congruence of residual generator and trace boundary in $\mathsf{M}_{\mathrm{fut}}$.** For any expressions $e, e' \in \mathrm{Expr}(V, X, \kappa)$ and environment $\rho$, unfolding evaluation gives

$$\mathrm{ev}_{\rho}(\delta_{e \ominus e'} - (\delta_e - \delta_{e'})) = (\mathrm{ev}_{\rho}(e) - \mathrm{ev}_{\rho}(e')) - (\mathrm{ev}_{\rho}(e) - \mathrm{ev}_{\rho}(e')) = 0, \tag{S6.2}$$

so $z_0 := \delta_{e_j \ominus e'_j} - (\delta_{e_j} - \delta_{e'_j}) \in \mathcal{Z}_{\mathrm{sem}}$. In the direct-sum carrier $\mathcal{W} = \bigoplus_m \mathcal{W}_m$ of S5.4, each single-step action $S_m : \mathcal{W}_m \to \mathcal{W}_{m+1}$ composes `liftMap` with packet and language transports, so Lemma S6.1(ii)(iii) yields $S_m(\mathcal{Z}_{\mathrm{sem}}) \subseteq \mathcal{Z}_{\mathrm{sem}}$. Inducting on the length of $a \in \{*\}^{<\omega}$: at $a = [\,]$, $\iota_j(z_0)$ lies in $\mathcal{Z}_{\mathrm{sem}}$ at site $j$ and vanishes elsewhere; if every site coordinate of $S_a(\iota_j(z_0))$ lies in $\mathcal{Z}_{\mathrm{sem}}$, applying $S$ once more keeps every site coordinate in $\mathcal{Z}_{\mathrm{sem}}$, whence Lemma S6.1(i) gives $D_m(S_a(\iota_j(z_0))) = U_{\rho_m,\varepsilon_m}(\cdot) = (0, 0)$ at every site $m$. Thus $\iota_j(z_0) \in K_{\mathrm{fut}}$, identifying both words in $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$.

**(2) Writable request in canonical next and dual whole-ledger destinations.** At cofinal stop $j$, S1.3 and S5.3 show that the selected completion receipt invokes `StockTarget.compiles_settled` and `birthProgram` at settlement; unfolding the admission successor identifies the actual next admission current at packet $j + 1$ with that target's `nextCurrent`. Transporting the entire paid-material tuple (environment, AST, paid state, trace, and bearer) along that full-current identity produces `rootedMaterial` and the writable native request `rootedRequest` inside packet $j + 1$, matching the source boundary's $\mathsf{M}_{\mathrm{fut}}$ class by (1) while supplying both `originalOldRow` and `originalBornRow` destinations from the same first whole-ledger write. ■

**Formal correspondence.** MV4's `RC/Semantics.lean` (`semanticZero`, `universal_inventory_zero`, `lift_semantic_zero`, `residual_semantic_zero`, `all_word_site_invisible`, `semantic_model_equiv`), `RC/Source.lean` (`rootedMaterial`, `rootedRequest`, `rooted_request_model`, `actual_generated_current`, `original_compiles`, `originalOldRow`, `originalBornRow`), and `RC/Consumer.lean` (`actual_root_future`, `actual_root_eliminate`); CP121.

## S6.2 Theorem 10.2 (C47–C49): regrouped residual evaluation, shared provenance, and same-debt settlement

**(1) Regrouped evaluation on full retained residuals (C47).** On the two-layer paired group $(V \times V) \times (V \times V)$, define the additive homomorphism

$$\mathrm{regroup}((u_{11}, u_{12}), (u_{21}, u_{22})) := ((u_{11} + u_{21},\, 0),\, (u_{12} + u_{22},\, 0)).$$

For any two-layer environment $\Theta$, put $\rho_{\mathrm{tot}}(x) := \Theta(x)_1 + \Theta(x)_2 \in V \times V$, $\rho_{\mathrm{phys}}(x) := (\pi_1(\rho_{\mathrm{tot}}(x)), 0)$, and $\rho_{\mathrm{res}}(x) := (\pi_2(\rho_{\mathrm{tot}}(x)), 0)$. Under binding $\beta(x) = \mathrm{linear}_{\mathrm{regroup}}(\mathrm{var}\, x)$, pointwise inspection gives $\mathrm{ev}_\Theta(\beta(x)) = (\rho_{\mathrm{phys}}(x), \rho_{\mathrm{res}}(x))$. Setting $\mathrm{jointCode}(c) := (\widetilde{c})[\beta]$ for $c \in \mathrm{Expr}(V \times V)$, substitution evaluation and (S0.1) yield

$$\mathrm{ev}_\Theta(\mathrm{jointCode}(c)) = \mathrm{ev}_{(\rho_{\mathrm{phys}}, \rho_{\mathrm{res}})}(\widetilde{c}) = (\mathrm{ev}_{\rho_{\mathrm{phys}}}(c),\ \mathrm{eff}_{\rho_{\mathrm{phys}}, \rho_{\mathrm{res}}}(c)), \tag{S6.3}$$

whose coordinates sum by Proposition 3.3 to $\mathrm{ev}_{\rho_{\mathrm{phys}} + \rho_{\mathrm{res}}}(c)$. Updating only `calculationReader` prior to emission preserves the query reader, decoder, and all four stock callbacks across every cofinal stop via S5.2 and S5.3.

**(2) Shared-provenance raw input and linear receiver continuation (C48).** Co-installing complete material and action-reader output $\mathrm{Raw} = (\rho_{\mathrm{Raw}}, e_{\mathrm{Raw}})$ under one projection restriction, define $\delta := \rho_{\mathrm{Born}} - \rho_{\mathrm{Raw}}$ from the born frame's active environment $\rho_{\mathrm{Born}}$, giving $\rho_{\mathrm{Raw}} + \delta = \rho_{\mathrm{Born}}$. Evaluating $\widetilde{e_{\mathrm{Raw}}}$ in $(\rho_{\mathrm{Raw}}, \delta)$ sums by (S0.1) and Proposition 3.3 to $\mathrm{ev}_{\rho_{\mathrm{Born}}}(e_{\mathrm{Raw}})$; in a linear homomorphic receiver epoch, re-reading the common source at each occurrence evaluates initial result plus actual effect.

**(3) Strict same-debt syntax debit and completed answer delivery (C49).** At each actual current, the selected receipt supplies a settled constant endpoint, so S2.1(1) equates the next writable residual's evaluation to $\mathrm{eff}_{\rho,\varepsilon}(e)$. Because the born frame executes step 1 at admission, remaining syntax credit $B - 1 = r(e) + r(e') + 1 \geq 1$ (S2.1(2)) drives strict per-step same-debt debits without refill across every paid branch—even when $\mathrm{eff}_{\rho,\varepsilon}(e) = 0$—eliminating dependently back to the identical ledger, visit, row, and whole-ledger destination. The clock-suffix endpoint lands at the completed current with value $\mathrm{eff}_{\rho,\varepsilon}(e)$, where `StockTarget.compiles_settled` delivers the completed answer, receipt, old/born row destinations, and canonical direct successor. ■

**Formal correspondence.** MV4's `SF/Factory/{Source,Consumer}.lean`, `SF/ActualInput/{Source,Consumer}.lean`, `SF/FullResidual/{Source,Consumer,Dynamics/Source,Dynamics/Consumer,Dynamics/Initial}.lean` (CP122–126); `Q/{Source,Consumer,Next/Source,Next/Consumer,Next/EnvironmentUpdate/...,Next/Linear/Receiver/...}.lean` (CP128–131, 135–136); and `SF/ActualInput/EnvironmentUpdate/CurrentMaterial/{Source,Consumer,RootedEffect/...}.lean` (CP132–134).

## S6.3 Theorem 10.3 (C51): ring-free double-negation extraction and cross-cofinal homomorphic continuation

**Objects and construction.** Fix an additive group family $W$ (no ring multiplication assumed) and pre-selected additive homomorphism $f : W_s \to W_s$. Define recursively $V_0(t) = W(t)$, $V_{n+1}(t) = V_n(t) \times V_n(t)$ and $f^{(0)} := f$, $f^{(n+1)}(u, v) := (f^{(n)}(u), f^{(n)}(v))$. On level-$n$ frame $\Phi$, define `retainedCode` by (10.3) and set the method input to evaluate $\mathrm{linear}_{f^{(n+1)}}(\widetilde{\mathrm{retainedCode}_n(\Phi)})$ in zero-increment paired environment $(\rho_\Phi, 0)$.

**Key lemma.**

**Lemma S6.3 (Zero-increment paired evaluation and double-negation identity).** On any additive group carrier $U$, environment $\rho$, and expression $e$: (i) $\mathrm{eff}_{\rho,0}(e) = 0$, whence $\mathrm{ev}_{(\rho,0)}(\widetilde{e}) = (\mathrm{ev}_\rho(e), 0)$; (ii) $\mathrm{ev}_\rho(\mathrm{linear}_{-\mathrm{id}}(\mathrm{linear}_{-\mathrm{id}}(e))) =$

$\mathrm{ev}_\rho(e)$; and (iii) across all paid-prefix steps $0 \le h \le d_0$ of Theorem 6.12, $\Phi_h$.registered.input.expression $= \Phi_0$.registered.input.expression.

*Proof.* (i) Setting $\varepsilon = 0$ in Proposition 3.3 gives $\mathrm{ev}_\rho(e) = \mathrm{ev}_\rho(e) + \mathrm{eff}_{\rho,0}(e)$, cancelling to $\mathrm{eff}_{\rho,0}(e) = 0$ and substituting into (S0.1). (ii) Unfolding two negations applies $-(-x) = x$. (iii) Inducting on $h$, each $h < d_0$ takes $\Phi_{h+1} = \Phi_h$.mathNext, which advances only the debt state while preserving registered input. ■

**Decisive derivation.** For any admission packet $P_n = (n, (\Phi_{\mathrm{src}}, \mathrm{seed}), \mathrm{cfg}, \text{language equality})$, Theorem 6.10(3) settles at constant endpoint $e'_{\mathrm{paid}} = \mathrm{const}(p)$. Triggering `nextBorn` at the end of the paid prefix sets the born frame's registered expression to $\mathrm{add}(e_{\mathrm{prev}}, \mathrm{linear}_{-\mathrm{id}}(e'_{\mathrm{paid}}))$ (since Lemma S6.3(iii) keeps $e_{\mathrm{prev}}$ unchanged throughout the prefix). Consequently, at the next native packet, pattern-matching `retainedCode` hits paid $= \mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(p))$ and outputs

$$\mathrm{retainedCode}_{n+1} = \mathrm{linear}_{-\mathrm{id}}(\mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(p))). \tag{S6.4}$$

Evaluating in $\rho_{\mathrm{next}}$ via Lemma S6.3(ii) yields $\mathrm{ev}_{\rho_{\mathrm{next}}}(\mathrm{retainedCode}_{n+1}) = -(-p) = p$. Applying Lemma S6.3(i) and $f^{(n+1)}(0) = 0$ then evaluates the next packet's action reader to

$$f^{(n+2)}(\mathrm{ev}_{(\rho_{\mathrm{next}},0)}(\widetilde{\mathrm{retainedCode}}_{n+1})) = (f^{(n+1)}(p),\ f^{(n+1)}(0)) = (f^{(n+1)}(p),\ 0). \tag{S6.5}$$

Meanwhile, setting `nextPairInventory` to `Stock.preserve` adjoins `completeQueryWritten` via the right parallel-unfolding trace inclusion (Theorem 2.2), placing every event of the complete query trace into `writtenStock`. Embedding the four-face projection component prior to query/result/compilation projections makes `rootRead` recover four faces, dual readback, and cochains definitionally across all cofinal stops and finite field segments. ■

**Formal correspondence.** MV5's `SF/ActualInput/PaidMethod/Source.lean` (`pairAction`, `actionAt`, `retainedCode`, `raw`, `raw_value`, `completePairOutput_query`, `templateFourFaces`), `PaidMethod/Recovery.lean` (`negate_paid`, `paid_prefix_expression`, `after_native_retained`, `after_native_value`, `after_native_whole_action`, `actual_complete_query_written`, `actual_native_faces`), and `PaidMethod/Admissions.lean` (`cofinal_paid_method_faces`, `finite_paid_method_faces`); CP483.

## S6.4 Theorem 10.4 (C52): primitive-by-primitive native replay and four-way cochain lockstep

**Objects and construction.** Given source process $\mathcal{P}$ and $\mathrm{readEnv} : S \to \mathrm{Env}$, define on $w \in \mathbb{Z}[S]$ the observer environment $\rho^{\mathrm{obs}}_w(x) := \widehat{\mathrm{readEnv}(\cdot)}(x)(w)$; at basis points $w = \delta_{s_t}$, Proposition 2.5 gives $\rho^{\mathrm{obs}}_{\delta_{s_t}} = \mathrm{readEnv}(s_t)$, and structural induction on $e$ yields $\mathrm{ev}_{\mathrm{ctx}(w)}(\mathrm{embed}(e)) = \mathrm{ev}_{\rho^{\mathrm{obs}}_w}(e)$.

**(1) Step-wise embedding and substitution charge.** Inducting on $s : e_1 \to e_2$ under $\rho^{\mathrm{obs}}_w$ defines `embeddedStep`: variable binds invoke the canonical execution of $\mathrm{embed}(\mathrm{var}\, x)$ rewritten by `embed_eval`; the three root constant contractions map to single-step traces; the five congruence rules lift subtraces recursively. Concatenation gives `embeddedTrace`; substituting along $\mathrm{ev}_{\mathrm{ctx}(w)} \circ \sigma = \mathrm{ctx}(Aw)$ on $Aw$ defines `forward(w,e)` and zero-effect lift `pairedForward(w,e)`, whose constant endpoint fixes its length via Proposition 3.7(1) at $r(\mathrm{embed}(e)[\sigma])$.

**(2) Born-stock write and next-query retention.** Inductive clock alignment equates frame $k$'s active environment with step $11 + k$ of the physical runtime, matching the state-point environment to $\tilde{\rho}_k$. Callback preservation writes `replayStock` into `birthStock`, which left parallel-seed inclusion carries into `nextBorn`'s `completeQueryWritten`.

**(3) Four-way lockstep identities.** Let $\mu_k$ be the source replay relation word with $w_{\mathrm{src},k} = \mathrm{rel}(\mu_k)$, $\tau_1$ the substitution chain map, and $b_{\mathrm{act},k} := \mathrm{rel}_{\tilde{\rho}_k}(\mathrm{nativeReplayAt}_k.\mathrm{relationWords})$ in the next relation closure. Telescoping endpoint differences (S0.3) and commuting substitution across basis differences gives $b_{\mathrm{act},k} = \mathrm{subst}_\sigma(w_{\mathrm{src},k})$. Applying the substitution boundary square $\mathrm{rel}_{\tilde{\rho}_k} \circ \tau_1 = \mathrm{subst}_\sigma \circ \mathrm{rel}$ to $\mu_k$ yields

$$\mathrm{rel}_{\tilde{\rho}_k}(\tau_1(\mu_k)) = \mathrm{subst}_\sigma(w_{\mathrm{src},k}) = b_{\mathrm{act},k}. \tag{S6.6}$$

In the natural cochain complex of the next prefix history, degree $0 \to 1$ differential $d^{0,1} : X^0 \hookrightarrow X^1$ is submodule inclusion from the relation closure into the generator closure, so on underlying words $d^{0,1}(b_{\mathrm{act},k}) = b_{\mathrm{act},k} = \mathrm{rel}_{\tilde{\rho}_k}(\tau_1(\mu_k))$ while completion projection $\pi_{\mathrm{comp}}$ on $X^1$ kills the relation closure: $\pi_{\mathrm{comp}}(d^{0,1}(b_{\mathrm{act},k})) = 0$. Finally, the paired binding square $\tilde{\rho}_k \circ \sigma = \tilde{\rho}_{k+1}$ makes $(\mathrm{subst}_\sigma, \mathrm{id})$ a morphism from count $k+1$'s root evaluation to count $k$'s, inducing by S0.4 $\mathrm{act}_{\mathrm{coim},k}([w_{\mathrm{src},k}]_{k+1}) = [\mathrm{subst}_\sigma(w_{\mathrm{src},k})]_k = [b_{\mathrm{act},k}]_k$, while `map_map` and `map_id` recover `rootRead` upon projecting the unfolding. ■

**Formal correspondence.** MV5's `Operations/Native/Context/WordExecution.lean` (`observerEnvironment`, `embed_eval`, `binding_word`, `embeddedStep`, `embeddedTrace`, `forward`, `pairedForward`, `forward_charge`) and `NM/RelationCochain/{Source,Consumer}.lean` (`replayStock`, `replay_written`, `replay_next_query`, `actual_boundary_from_source`, `actual_complex_boundary`, `generated_differential`, `actual_complex_differential`, `completion_consumes_relation`, `actual_coimage_boundary`, `actual_coimage_differential`, `actual_relation_cochain_method`); CP485.

## S6.5 Theorem 10.5 (C53 and C50 window stock note): bidirectional step–request slice inversion and anti-tamper gates

**Objects and batch expansion.** Mapping the $M$ paid-step relation words through $S_P$ forms relation-event list $E = [e_0, \ldots, e_{M-1}]$, where $e_i = \mathrm{relation}(S_P w_i)$ carries signed support items $\mathrm{items}(e_i)$ of length $c_i = |\mathrm{items}(e_i)|$. Request expansion from offset $s_0$ obeys `requestsFrom_append`: $\mathrm{req}(s_0, E_1 +\!\!+ E_2) = \mathrm{req}(s_0, E_1) +\!\!+ \mathrm{req}(s_0 + |E_1|, E_2)$. Write $\mathcal{B} := \mathrm{req}(s_0, E)$ of total length $N = |\mathcal{B}|$.

**(1) Forward contiguous slice (`stepInterval`).** For any step $i < M$, split $E = E[0 : i] +\!\!+ [e_i] +\!\!+ E[i+1 : M]$. Applying `requestsFrom_append` twice with $|E[0 : i]| = i$ decomposes $\mathcal{B} = \mathrm{prior}_i +\!\!+ \mathrm{own}_i +\!\!+ \mathrm{later}_i$, where $a_i := |\mathrm{prior}_i| = |\mathrm{req}(s_0, E[0 : i])|$ and $\mathrm{own}_i = \mathrm{req}(s_0 + i, [e_i])$ has length $|\mathrm{own}_i| = |\mathrm{items}(e_i)| = c_i$. By `(prior ++ own ++ later).drop |prior| |>.take |own| = own`, $\mathcal{B}[a_i : a_i + c_i) = \mathrm{own}_i$, whose request words sum under $\rho_{\mathrm{dec}}$ via `relations_request_sum` to $\mathrm{affine}_{\rho_{\mathrm{dec}}}(S_P w_i) := S_P w_i - q_{\rho_{\mathrm{dec}}}(S_P w_i)$.

**(2) Backward unique origin and address identity (`readRequestOrigin`).** Setting $P(n) := \mathrm{req}(s_0, E[0 : n])$ gives $P(0) = [\,]$, $P(n+1) = P(n) +\!\!+ \mathrm{part}(n)$, and $P(M) = \mathcal{B}$. For any $r < N = |P(M)|$, constructive monotone partition lookup `locatePart` returns the unique block

$i < M$ and local slot $s < |\mathrm{part}(i)| = c_i$ with $r = |P(i)| + s$ and $\mathcal{B}[r] = \mathrm{part}(i)[s]$. Cancelling the shared source-prefix length |before| proves

$$r = a_i + s, \qquad \mathcal{B}[0 : r] = \mathrm{req}(s_0, E[0 : i]) +\!+ \mathrm{map}_{\mathrm{form}}(\mathrm{items}(e_i)[0 : s]). \tag{S6.7}$$

**(3) Full-batch splice and anti-tamper negative-control gate.** Forming $G := [\mathcal{B}[a_0 : a_0 + c_0), \ldots, \mathcal{B}[a_{M-1} : a_{M-1} + c_{M-1})]$ (keeping empty slices $c_i = 0$ as [] in place), item (1) gives $G[i] = \mathrm{req}(s_0 + i, [e_i])$, whose list flatten recovers $\mathcal{B}$; homomorphism of `executedTrace` over grouped concatenation recovers executedTrace($\mathcal{B}$), and nested list-sum associativity recovers the full batch boundary. **Negative-control proof**: if step $i$ has a nonempty group ($c_i > 0$), each request contributes at least one execution event so its native trace slice has positive length $\ell_i > 0$; omitting or duplicating group $i$ changes total trace length to $L - \ell_i < L$ or $L + \ell_i > L$, strictly altering the native executed trace.

*Note on C50 (four-trace window stock and inverse-residual correction).* In the Riesz finite $P_a$-window instantiation (MV4, CP217/CP219), received Möbius source $\nu$ and both means compile cutoff pullbacks, reciprocal weights, and Tate poles into literal expressions prior to writeback admission so forward, inverse, second-mean, and cochain traces all enter born stock; reading the first inverse residual then executes $\nu -$ residual under cutoff substitution to adjoin the correction trace while preserving all four prior stock traces and full joint-observation fibres. ■

**Formal correspondence.** `7e4da746f3`'s `A_src/Receipt.lean`, `A_src/Receipt/{Origin,Block,Block/Position,Block/SourceOrigin,Block/StepInterval,Block/StepInterval/Whole}.lean` (`RequestOriginAt`, `readRequestOrigin`, `StepInterval`, `stepInterval`, `sourceRequestAt_interval_address`, `grouped_requests`, `grouped_executed_trace`, `grouped_request_boundary`), `NextReceiptStepJointWhole.lean`, and `Verification/no-island/riemann-r-cofinal-complete-grouped-write/Gate.lean` (CP514, 521, 528–532); windowed action in MV4 `J/ClozelOriginalK{FinitePaWindowOperator,WindowInverseResidual}SourceAction/...` (CP217, CP219).

## S6.6 Theorem 10.6 (C54): sealed-prefix rigidity, tail-only incremental continuation, and single-pass timeline

**(1) Sealed-history uniqueness and tail continuation (`extendTo_prefix`).** In inquiry runtime $\mathcal{R}$, structural induction on $H \in \mathsf{HistoryAt}(\mathcal{R}, \mathrm{fuel}, s_0)$ proves $H = \mathrm{runFrom}(\mathrm{fuel}, s_0)$: base `.exhausted` is `rfl`, and `.step activated tail` rewrites `tail` by the induction hypothesis to fold back to runFrom(fuel + 1, $s_0$). Inducting on fuel across `Fin.cases` (eliminating identity casts via `occurrenceAt_cast`) proves $H$.storedOccurrenceAt($i$) = $H$.occurrenceAt($i$).

Recursing on fuel defines `continueFor(H, extra)`: at fuel = 0 (the reached tail state $s_{\mathrm{tail}}$) it launches `runFrom(extra, s_tail)`; at fuel > 0 it keeps `.step activated` untouched and recurses solely on the tail, never recomputing any of the first fuel activations. Transporting `continueFor(H, total - fuel)` along (total − fuel) + fuel = total defines `extendTo(H, total)`, and uniqueness `eq_runFrom` with `prefix_runFrom` gives in one line:

$$\mathrm{prefix}_{\mathrm{fuel}}(\mathrm{extendTo}(H, \mathrm{total})) = \mathrm{prefix}_{\mathrm{fuel}}(\mathrm{runFrom}(\mathrm{total}, s_0)) = \mathrm{runFrom}(\mathrm{fuel}, s_0) = H. \tag{S6.8}$$

**(2) Single-pass timeline table and stage-inventory recovery (`storedTimeline`).** On micro-history $H_{\mathrm{mic}} \in \mathsf{SourceNativeLivingRootHistoryAt}(\mathcal{P}, \mathrm{fuel}, s_0)$ with $\pi : S \to M$, tail-recursive `collect` starts from an empty array pre-allocated to capacity $\mathrm{fuel}+1$, pushing each stored state projection once plus the terminal state to yield exact size $|T| = 0 + (\mathrm{fuel}+1) = \mathrm{fuel}+1$. Inducting over arbitrary initial accumulator size $|\mathrm{acc}|$ proves `collect_adjacent`: index 0 reads $(\pi(s_0), \pi(s_1))$, while index $j+1$ shifts $|\mathrm{acc}| + (j+1) = (|\mathrm{acc}|+1) + j$ to the tail hypothesis. Specializing to $|\mathrm{acc}| = 0$ yields $(T[i], T[i+1]) = (\pi(s_i), \pi(s_{i+1}))$ for all $i <$ fuel, so applying additive $\rho \circ \pi$ equates $(\rho(T[i]),\ \rho(T[i+1]) - T[i]))$ to $(\rho_i, \varepsilon_i)$ and matches `timelineInventory` to $U_i^{s_0}$ of Definition 4.5. ■

**Formal correspondence.** `7e4da746f3`'s `Operations/Inquiry/NativeHistoryContinuation.lean` (`HistoryAt.eq_runFrom`, `storedOccurrenceAt`, `stored_occurrence_eq`, `continueFor`, `extendTo`, `extendTo_prefix`), `Operations/Runtime/StoredHistory.lean` (`stored_pair_actual`, `stored_environments_actual`, `stored_inventory_actual`), and `Operations/Runtime/StoredTimeline.lean` (`storedTimeline`, `stored_timeline_adjacent`, `timelineEnvironments`, `timelineInventory`, `timeline_environments_actual`, `timeline_inventory_actual`), with Goldbach micro-actor/timeline gates (CP596–598).

## S7 Complete claim-to-proof index (C1–C54)

The table maps every claim **C1–C54** (together with the T0 single-constructor kernel, auxiliary domain bridges C14–C21, and cross-domain instantiations) to its exact Third-Edition (v3) main-text result, self-contained written proof, and pinned source commit.

| Claim ID | Third-Edition (v3) main-text location | Self-contained written proof | Pinned commit |
|---|---|---|---|
| T0 unfolding & root-read kernel | §2.1 Def. 2.1, Thm. 2.2 | Main text §2.1 (complete) | H / MV4 (`[propext]` for parallel cardinality, `[]` for the other declarations) |
| C1 | §6.1 Prop. 6.1 | Main text §6.1 (complete) | H (`8e29b8e` alignment) |
| C2 | §6.4 Def. 6.6, Thm. 6.7, Note 6.8, Fig. S2 | Main text §6.4 (complete) | H (wf kernel `[]`) |
| C3 | §2.2 Def. 2.3, Prop. 2.4 | Main text §2.2 (complete) | H (`[]`) |
| C4 | §8.4 canonical inquiry runtime activation | Main text §8.4 | H |
| C5 | §8.4 Prop. 8.9, Table 4, Fig. 5 | Main text §8.4; App. C.3 | H |
| C6 | §8.4 Prop. 8.10 | Main text §8.4 (complete) | H (`[]`) |
| C7 | §3.1–§3.3 Defs. 3.1–3.2, 3.4, Props. 3.3, 3.5–3.6 | Main text §3.1–§3.3 (complete) | H |
| C8 | §4 Thms. 4.4, 4.6–4.7, Prop. 4.8, Exs. 4.1–4.2, Figs. 1, 3–4, S1 | Main text §4; Apps. A–B | H |
| C9 | §2.3–§2.4 Prop. 2.5, Def. 2.6, Prop. 2.7 | Main text §2.3–§2.4; App. A.1 | H |
| C10 | §5 Def. 5.1, Prop. 5.2, Thm. 5.3 | Main text §5 (complete) | H (`[]`) |

| Claim ID | Third-Edition (v3) main-text location | Self-contained written proof | Pinned commit |
|---|---|---|---|
| C11 | §7.1–§7.2 Prop. 7.1, Def. 7.2, Props. 7.3–7.5 | Main text §7; App. C.6 | H (7.3–7.4 `[]`) |
| C12 | §6.2–§6.3 Defs. 6.2–6.3, Props. 6.4–6.5, Table 2 | Main text §6.2–§6.3; App. C.5 | H (6.4–6.5 `[]`) |
| C13 | §8.1–§8.3 Def. 8.1, Props. 8.2–8.3, Thm. 8.4, Def. 8.5, Thms. 8.6–8.7, Prop. 8.8 | Main text §8.1–§8.3; Apps. C.1–C.4 | H (`[]`) |
| C14–C18 (upstream & cross-paper bridges) | C14–C15 gated/dormant auxiliary kernels (non-main claims); C16 feeds §7.2 Prop. 7.5; C17 is §7.1 same-debt control; C18 belongs to *Exact Realization* R14–R17 | Prop. 7.5 in App. C.6; others in upstream sources & companion paper | H / X / Z |
| C19–C21 (generalized into main chain) | Subsumed into §6.5 Thm. 6.10, §8.5 Thm. 8.12 & §10.4 Fock instantiation | Supplement S1, S2.1 | AA / AB |
| C22 | §6.5 Def. 6.9, Thm. 6.10 | Supplement S1 | AB |
| C23 | §3.4 Prop. 3.7, Def. 3.8, Thm. 3.9; §6.5 Prop. 6.11; §10.4 | Main text §3.4, §6.5 (complete) | AB |
| C24 | §8.5 Def. 8.11, Thm. 8.12, Ex. 8.13, Fig. 6 | Supplement S2.1 | AB |
| C25 | §7.2 Prop. 7.6 | Main text §7.2 (complete) | AB |
| C26–C27 | §7.2 Thm. 7.7(1)–(3) | Supplement S2.2 | AC |
| C28 | §9.1 Def. 9.1, Thm. 9.2 | Supplement S3.1 | AD |
| C29 | §9.2 Thm. 9.3 | Supplement S3.2 | AD |
| C30 | §9.3 Def. 9.4, Thm. 9.5 | Supplement S4.1 | AD |
| C31 | §9.3 Thm. 9.6 | Supplement S4.2 | AE |
| C32 | §9.3 Thm. 9.7 | Supplement S4.3 | AE |
| C33–C34 | §9.4 Thm. 9.8(1)–(3) | Supplement S5.1 | AE |
| C35–C38 | §8.5 Thm. 8.14 (boundary difference, own fee & environment square) | Supplement S2.3 | PR, MF, H0-S |
| C39 | §6.5 Thm. 6.12 | Supplement S1.3 | MV2 / MV3 |
| C40 | §9.4 Thm. 9.9, Ex. 9.13(1) | Supplement S4.4 | MV2 / MV3 |
| C41 | §10.4 resolvent comb $R^m q_0$ calculation, Fig. 7B | Supplement S5.5 | MV2 / MV3 |
| C42 | §8.5 Thm. 8.15; §10.4 comb total-value feedback, Fig. 7 | Supplement S5.2, S5.5 | MV3 |
| C43 | §9.4 Thm. 9.10, Ex. 9.13(2) | Supplement S5.3 | MV3 |
| C44 | §9.4 Thm. 9.11, Ex. 9.13(3) | Supplement S3.3 | MV3 |
| C45 | §9.4 Thm. 9.12, Ex. 9.13(3) | Supplement S5.4 | MV3 |
| C46 | §10.1 Thm. 10.1, Fig. 8A | Supplement S6.1 | MV4 |
| C47–C49 | §10.1 Thm. 10.2(1)–(3), Fig. 8A | Supplement S6.2 | MV4 |

| Claim ID | Third-Edition (v3) main-text location | Self-contained written proof | Pinned commit |
|---|---|---|---|
| C50 | §10.4 finite $P_a$-window 4-trace stock & inverse-residual correction | Main text §10.4; Supp. S6.5 note | MV4 |
| C51 | §10.2 Thm. 10.3, Ex. 1.1(b), Ex. 10.7, Fig. 8B | Supplement S6.3 | MV5 |
| C52 | §10.2 Thm. 10.4, Fig. 8B | Supplement S6.4 | MV5 |
| C53 | §10.3 Thm. 10.5, Ex. 10.7, Fig. 8C | Supplement S6.5 | `7e4da746f3` |
| C54 | §10.3 Thm. 10.6, Ex. 10.7, Fig. 8C | Supplement S6.6 | `7e4da746f3` |
| CPS1 37-primitive body | §10.4 Group 3 whole-atom smoothness & reserve dispatch | Supplement S5.6 | MV3 |